NODE.JS

CHEAT SHEETS

A Complete Reference for Server-Side JavaScript

Modules & npm · Event Loop · Async Patterns · File System · Streams · HTTP & fetch · Process & CLI · Testing · Worker Threads · Security · Express & Fastify · Deployment

70 REFERENCE SHEETS

CONTENTS

INTRODUCTION & SETUP
01 Introduction & Architecture
02 Installation & Version Managers
03 Running Node
04 Node vs the Browser
05 TypeScript in Node

MODULES & PACKAGES
06 CommonJS Modules
07 ES Modules
08 CJS ↔ ESM Interop
09 package.json
10 npm CLI
11 pnpm, Yarn & Workspaces

EVENT LOOP & ASYNC
12 The Event Loop
13 Timers
14 Callbacks & Error-First Style
15 Promise-Based Core APIs
16 Async Patterns & Cancellation
17 EventEmitter

PROCESS & ENVIRONMENT
18 The process Object
19 Signals & Graceful Shutdown
20 Environment Variables & Config
21 The os Module

BUILDING CLI TOOLS
22 Command-Line Arguments
23 Reading Input
24 Console & Terminal Output

FILE SYSTEM
25 fs API Styles
26 Reading Files
27 Writing & Appending
28 Directories & Globbing
29 File Info & Watching
30 The path Module

BUFFERS & STREAMS
31 Buffer Basics
32 Binary Data
33 Stream Concepts
34 Readable & Writable Streams
35 pipeline() & Transforms
36 Web Streams

NETWORKING & HTTP
37 HTTP Server Basics
38 Requests, Bodies & URLs
39 fetch & Undici
40 HTTPS & TLS
41 WebSockets
42 TCP, UDP & DNS

ERRORS & DEBUGGING
43 Error Patterns in Node
44 Uncaught Errors & Rejections
45 Debugging
46 Diagnostics & Tracing

TESTING
47 node:test Basics
48 The assert Module
49 Mocking, Snapshots & Coverage

CONCURRENCY & PERFORMANCE
50 Child Processes
51 Worker Threads
52 Cluster & Scaling
53 Measuring Performance
54 Memory & Garbage Collection

SECURITY
55 Permission Model
56 Crypto
57 Supply Chain & Hardening

BUILT-IN UTILITIES
58 The util Module
59 AsyncLocalStorage
60 node:sqlite

WEB FRAMEWORKS & LIBRARIES
61 Express Basics
62 Express Middleware & Routers
63 Fastify
64 Databases from Node
65 Logging

DEPLOYMENT
66 Running in Production
67 Docker for Node
68 Single Executables & Bundling

QUICK REFERENCE
69 Quick Reference — Runtime, Modules & Async
70 Quick Reference — Files, Network & Tooling

NODE

Table of Contents

70 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 05

- 01 Introduction & Archite…
- 02 Installation & Version …
- 03 Running Node
- 04 Node vs the Browser
- 05 TypeScript in Node

MODULES & PACKAGES 06 – 11

- 06 CommonJS Modules
- 07 ES Modules
- 08 CJS ↔ ESM Interop
- 09 package.json
- 10 npm CLI
- 11 pnpm, Yarn & Workspa…

EVENT LOOP & ASYNC 12 – 17

- 12 The Event Loop
- 13 Timers
- 14 Callbacks & Error-First…
- 15 Promise-Based Core A…
- 16 Async Patterns & Can…
- 17 EventEmitter

PROCESS & ENVIRONMENT 18 – 21

- 18 The process Object
- 19 Signals & Graceful Shu…
- 20 Environment Variables …
- 21 The os Module

BUILDING CLI TOOLS 22 – 24

- 22 Command-Line Argum…
- 23 Reading Input
- 24 Console & Terminal Ou…

FILE SYSTEM 25 – 30

- 25 fs API Styles
- 26 Reading Files
- 27 Writing & Appending
- 28 Directories & Globbing
- 29 File Info & Watching
- 30 The path Module

BUFFERS & STREAMS 31 – 36

- 31 Buffer Basics
- 32 Binary Data
- 33 Stream Concepts
- 34 Readable & Writable S…
- 35 pipeline() & Transforms
- 36 Web Streams

NETWORKING & HTTP 37 – 42

- 37 HTTP Server Basics
- 38 Requests, Bodies & UR…
- 39 fetch & Undici
- 40 HTTPS & TLS
- 41 WebSockets
- 42 TCP, UDP & DNS

ERRORS & DEBUGGING 43 – 46

- 43 Error Patterns in Node
- 44 Uncaught Errors & Rej…
- 45 Debugging
- 46 Diagnostics & Tracing

TESTING 47 – 49

- 47 node:test Basics
- 48 The assert Module
- 49 Mocking, Snapshots & …

CONCURRENCY & PERFORMANCE 50 – 54

- 50 Child Processes
- 51 Worker Threads
- 52 Cluster & Scaling
- 53 Measuring Performance
- 54 Memory & Garbage Co…

SECURITY 55 – 57

- 55 Permission Model
- 56 Crypto
- 57 Supply Chain & Harde…

BUILT-IN UTILITIES 58 – 60

- 58 The util Module
- 59 AsyncLocalStorage
- 60 node:sqlite

WEB FRAMEWORKS & LIBRARIES 61 – 65

- 61 Express Basics
- 62 Express Middleware & …
- 63 Fastify
- 64 Databases from Node
- 65 Logging

DEPLOYMENT 66 – 68

- 66 Running in Production
- 67 Docker for Node
- 68 Single Executables & B…

QUICK REFERENCE 69 – 70

- 69 Quick Reference — Ru…
- 70 Quick Reference — File…

Sheet 01

Introduction & Architecture

V8 · libuv · single thread + thread pool · non-blocking I/O · use cases · release schedule & LTS

WHAT NODE.JS IS

Node.js is a JavaScript runtime that runs outside the browser. It pairs Google's **V8** engine (which executes JavaScript) with **libuv** (which handles files, sockets, timers, and threads). Your code runs on a **single main thread**; slow I/O is handed off and its callback runs later, so one process can serve thousands of connections without waiting. Created by Ryan Dahl in 2009, now maintained under the OpenJS Foundation.

NOTES

- File types: `.js` · `.mjs` (ESM) · `.cjs` (CommonJS) · `.ts`
- Node 26 enters LTS in October 2026
- From Node 27: one major release a year, every release becomes LTS
- CPU-heavy work blocks the loop; move it to `worker_threads`

HELLO, WORLD & NON-BLOCKING I/O

```
// app.mjs — run with: node app.mjs
import { readFile } from 'node:fs/promises';
import { createServer } from 'node:http';

console.log('Hello, World!');              // Hello, World!

console.log('start');                      // printed 1st
readFile('notes.txt', 'utf8')              // handed off to libuv
  .then(text => console.log(text.length)); // printed 3rd, when the read finishes
console.log('end');                        // printed 2nd, nothing waited

createServer((req, res) => res.end('Hi\n'))
  .listen(3000, () => console.log('http://localhost:3000'));
```

CODE EXPLAINED

- → `node:` prefix — marks a built-in core module
- → `readFile` — returns at once; result arrives as a promise
- → `'end'` before length — the loop never blocked on disk
- → `createServer` — one callback handles every request
- → `listen()` — keeps the process alive until closed

ARCHITECTURE LAYERS

LAYER	ROLE
Your code	JS / TS modules
Core modules	fs, http, stream, crypto…
Bindings	C++ glue to JS
V8	Compiles & runs JS
libuv	Event loop, async I/O
Thread pool	4 threads: fs, dns, crypto, zlib

RELEASE PHASES

PHASE	LENGTH	GETS
Current	~6 mo	New features
Active LTS	~12 mo	Fixes, backports
Maintenance	~18 mo	Security, critical
End of life	—	Nothing

GOOD FIT / POOR FIT

USE	FIT
APIs & web servers	✔
Real-time (chat, sockets)	✔
CLI tools & scripts	✔
Build tooling	✔
Heavy number crunching	✖
Video encoding in-process	✖

COMMON MISTAKES

- Assuming "single-threaded" means slow: only your JS is single-threaded; I/O runs in parallel
- Running a long loop in a request handler; every other request waits until it ends
- Using an odd-numbered or EOL version in production; they stop getting security fixes

TIPS

- Run production on an Active LTS line; test on Current in CI
- Always import core modules as `node:fs`, not `fs`
- Raise the pool with `UV_THREADPOOL_SIZE` for heavy fs or crypto work

SUMMARY

- `V8` — runs your JavaScript
- `libuv` — event loop and async I/O
- `non-blocking` — start I/O, continue, handle result later
- `LTS` — the version line to deploy

Installation & Version Managers

nodejs.org installers · nvm · fnm · .nvmrc · node -v · npm -v · engines field

INSTALLING NODE

You can install Node from the official installer, a system package manager, or a **version manager**. A version manager is the best choice for development: it installs Node per user (no `sudo`), switches versions per project, and reads a pinned version from `.nvmrc` or `.node-version`. Every install includes **npm** and **npx**.

NOTES

- `nvm` is macOS / Linux only; `nvm-windows` is a separate project
- `fnm` works on all three OSes and is faster
- Global npm packages belong to one Node version each
- Get nvm's install command from github.com/nvm-sh/nvm

VERSION MANAGER WORKFLOW

```
nvm install --lts            # newest LTS release
nvm install 26               # newest 26.x
nvm use 24                   # switch this shell to 24.x
nvm alias default 26         # version for new shells
nvm ls                       # list installed versions
echo "26" > .nvmrc && nvm use   # pin, then use the project version

fnm install 26 && fnm use 26  # same idea with fnm
eval "$(fnm env --use-on-cd)"  # auto-switch when you cd into a project

node -v                      # v26.x.x
npm -v                       # bundled npm version
which node                   # path of the active binary (where on Windows)
```

CODE EXPLAINED

- → `--lts` — tracks the newest LTS line
- → `use` — affects the current shell only
- → `alias default` — persists across terminals
- → `.nvmrc` — commit it so the team matches
- → `--use-on-cd` — put in your shell profile
- → `which node` — confirms which install wins on PATH

INSTALL METHODS

METHOD	COMMAND / SOURCE
`Installer`	nodejs.org (LTS button)
`Homebrew`	brew install node@24
`winget`	winget install OpenJS.NodeJS.LTS
`nvm / fnm`	per-user, many versions
`Docker`	FROM node:26-slim

FILES THAT PIN A VERSION

FILE	READ BY
`.nvmrc`	nvm, fnm
`.node-version`	fnm, many CI tools
`.tool-versions`	asdf, mise
`engines`	npm (warning)
`Dockerfile`	image tag

PACKAGE.JSON ENGINES

```
"engines": {
  "node": ">=24"
}
```

COMMON MISTAKES

- Installing with `sudo apt install nodejs`: distro packages are often several majors behind
- Mixing a system Node and nvm; whichever is first on PATH wins silently
- Using `sudo npm i -g`; fix permissions with a version manager instead

TIPS

- Commit `.nvmrc` and use the same file in CI setup
- Add `engine-strict=true` to `.npmrc` to make `engines` an error
- Reinstall globals on upgrade: `nvm install 26 --reinstall-packages-from=24`

SUMMARY

- `nvm / fnm` — install and switch versions
- `.nvmrc` — pins the project's version
- `node -v` — verify what's active
- `engines` — declares supported versions

Running Node

WAYS TO RUN CODE

The `node` command runs a file, evaluates a string, or opens the interactive **REPL** (Read-Eval-Print Loop). Built-in flags now cover jobs that used to need extra packages: `--watch` replaces nodemon, `--env-file` replaces dotenv, and `--run` runs `package.json` scripts faster than `npm run`.

NOTES

- Flags go **before** the file name; args after it go to `process.argv`
- `node --run` skips `pre` / `post` hooks
- Set default flags for every run with `NODE_OPTIONS`
- `node --help` lists every flag

COMMON INVOCATIONS

```
node app.js                       # run a script
node app.js --port 8080           # extra args land in process.argv
node                              # open the REPL (Ctrl+D to quit)
node -e "console.log(1 + 1)"      # evaluate: prints 2
node -p "2 ** 10"                 # evaluate and print: 1024
node --watch app.js               # restart when imported files change
node --watch-path=src app.js      # watch a specific folder
node --env-file=.env app.js       # load KEY=value pairs into process.env
node --env-file-if-exists=.env.local app.js   # no error if missing
node --run dev                    # run the "dev" script from package.json
node --check app.js               # syntax check only, no execution

#!/usr/bin/env node           # shebang: first line of an executable script
chmod +x cli.js && ./cli.js   # run it directly (macOS / Linux)
```

CODE EXPLAINED

- → `-e` — runs code, prints nothing by itself
- → `-p` — runs code and prints the result
- → `--watch` — watches the entry file and its imports
- → `--env-file` — repeatable; later files win
- → `--run` — adds `node_modules/.bin` to PATH
- → `shebang` — lets the OS find node on PATH

REPL COMMANDS

COMMAND	DOES
`_`	Last result
`_error`	Last thrown error
`.editor`	Multi-line mode
`.load file`	Run a file in session
`.save file`	Save session input
`Tab`	Autocomplete
`.exit`	Quit (or Ctrl+D)

USEFUL FLAGS

FLAG	PURPOSE
`--import mod`	Preload an ES module
`-r mod`	Preload a CJS module
`--inspect`	Enable the debugger
`--enable-source-maps`	Map stack traces
`--max-old-space-size`	Heap limit in MB
`--no-warnings`	Hide process warnings
`-v`	Print version

EXIT CODES

CODE	MEANING
`0`	Success
`1`	Uncaught exception
`9`	Invalid argument / flag
`13`	Unsettled top-level await
`130`	Ctrl+C (SIGINT)
`143`	SIGTERM

COMMON MISTAKES

- Putting flags after the file: `node app.js --watch` passes `--watch` to your app instead
- Using `-e` and expecting output; use `-p` to print
- Expecting `--watch` to see files you read with `fs`; it tracks imports only

TIPS

- Dev script: `"dev": "node --watch --env-file=.env app.js"`
- Use `node -p` for quick math and JSON checks in the shell
- Check scripts in CI with `node --check` before deploying

SUMMARY

- `node file` — run a script
- `node` — interactive REPL
- `--watch` — restart on change
- `--env-file` — load env vars
- `--run` — run a package script

Node vs the Browser

SAME LANGUAGE, DIFFERENT HOST

Node runs the same JavaScript as the browser but provides a different set of globals. There is **no DOM**, no `window`, and no `document`. Instead you get `process`, `Buffer`, and the core modules for files, networking, and the OS. Node also implements most standard **Web APIs** (`fetch`, URL, `AbortController`, Web Crypto), so a lot of browser code runs unchanged.

NOTES

- Use `globalThis` for code that runs in both
- `__dirname` exists in CommonJS only
- The `node:` prefix is required for some modules (`node:test` , `node:sqlite`)
- Node has full disk and network access; there is no sandbox by default

GLOBALS & WEB APIS IN NODE

```
import { readFile } from 'node:fs/promises';   // core module, node: prefix

typeof window;                          // 'undefined'
globalThis === global;                  // true
process.version;                        // 'v26.x.x'
import.meta.filename;                   // '/app/src/index.js'  (ESM)
import.meta.dirname;                    // '/app/src'           (ESM)
import.meta.url;                        // 'file:///app/src/index.js'

const res = await fetch('https://api.github.com/zen');   // Response
await res.text();                       // 'Keep it logically awesome.'
new URL('/a?b=1', 'https://x.dev').searchParams.get('b'); // '1'
structuredClone({ list: [1, 2] });   // deep copy
crypto.randomUUID();                    // '3b241101-e2bb-4255-8caf-…'
new TextEncoder().encode('hi');      // Uint8Array(2) [ 104, 105 ]
```

CODE EXPLAINED

- → `global` — Node's old name for `globalThis`
- → `import.meta.dirname` — ESM replacement for `__dirname`
- → `import.meta.url` — a `file://` URL, not a path
- → `await` at top level — allowed in ES modules
- → `crypto` — global Web Crypto, no import needed

BROWSER → NODE

BROWSER	NODE
`window`	globalThis
`document`	none (jsdom for tests)
`location.href`	import.meta.url
`localStorage`	files, node:sqlite
`prompt()`	readline/promises
`<script>`	import / require
`DevTools`	node --inspect

WEB APIS BUILT IN

API	USE
`fetch`	HTTP requests
`URL`	Parse / build URLs
`AbortController`	Cancel async work
`crypto.subtle`	Web Crypto
`WebSocket`	Socket client
`Blob`	Binary data
`performance`	High-res timing

NODE-ONLY GLOBALS

GLOBAL	USE
`process`	Env, args, exit
`Buffer`	Binary data
`global`	Alias of globalThis
`setImmediate`	Run after I/O
`require`	CJS only
`__dirname`	CJS only

COMMON MISTAKES

- Using `__dirname` in an ES module throws `ReferenceError` ; use `import.meta.dirname`
- Passing `import.meta.url` to `fs` as a path string; it's a URL
- Installing `node-fetch` or `uuid` for things Node now has built in

TIPS

- Check for Node at runtime: `typeof process !== 'undefined'`
- Convert URLs with `fileURLToPath(import.meta.url)` from `node:url`
- Prefer Web APIs when code may also run in browsers or edge runtimes

SUMMARY

- `globalThis` — the global object everywhere
- `process` — Node's runtime interface
- `import.meta` — current module's location
- `Web APIs` — fetch, URL, crypto built in

TypeScript in Node

type stripping · erasable syntax only · --experimental-transform-types · .ts / .mts imports · tsx · tsconfig for Node

RUNNING .TS FILES DIRECTLY

Node runs `.ts` files with no build step by **stripping types**: it replaces type annotations with whitespace and runs what's left. That only works for **erasable syntax**, TypeScript that disappears completely once types are removed. Node does **not type-check**; run `tsc --noEmit` (in your editor or CI) to catch type errors.

NOTES

- On by default since Node 23.6 and 22.18; no flag needed
- Imports must include the `.ts` extension
- Type-only imports need `import type`
- `.ts` files inside `node_modules` are not stripped

ERASABLE TYPESCRIPT

```ts
// greet.ts — run with: node greet.ts
import type { User } from './types.ts';   // type-only, with .ts extension

interface Options { loud?: boolean }

export function greet(user: User, opts: Options = {}): string {
  const msg = `Hello, ${user.name}`;
  return opts.loud ? msg.toUpperCase() : msg;
}

console.log(greet({ name: 'Ada' }));                  // Hello, Ada
console.log(greet({ name: 'Ada' }, { loud: true })); // HELLO, ADA

// ✖ Not erasable: these emit runtime code → SyntaxError
enum Color { Red, Green }
class Point { constructor(public x: number) {} }   // parameter property
```

CODE EXPLAINED

- → `import type` — removed entirely at runtime
- → `interface` — pure type, erased
- → `: string` — annotations become whitespace
- → `enum` — generates an object, so it can't be erased
- → `public x` — generates an assignment; same problem
- → Use a union or `as const` object instead of `enum`

WHAT STRIPS

SYNTAX	WORKS
types, interfaces	✔
generics, as, satisfies	✔
enum	✖
namespace (with code)	✖
parameter properties	✖
.tsx / JSX	✖
tsconfig paths	✖

WAYS TO RUN TS

COMMAND	WHEN
node app.ts	Erasable code
node --experimental-transform-types	Enums, etc.
npx tsx app.ts	Full TS + paths
tsc && node dist/app.js	Compiled build
tsc --noEmit	Type check only

TSCONFIG.JSON FOR NODE

```json
{
  "compilerOptions": {
    "module": "nodenext",
    "target": "esnext",
    "noEmit": true,
    "erasableSyntaxOnly": true,
    "verbatimModuleSyntax": true,
    "allowImportingTsExtensions": true
  }
}
```

COMMON MISTAKES

- Assuming Node checks types; a wrong type runs fine until it crashes
- Importing `./types` without `.ts`; Node won't guess the extension
- Importing a type without `import type`; the import stays and fails at runtime

TIPS

- Turn on `erasableSyntaxOnly` so `tsc` flags unsupported syntax
- Add `"check": "tsc --noEmit"` and run it in CI
- Libraries you publish should still ship compiled `.js` + `.d.ts`

SUMMARY

- `node app.ts` — strips types and runs
- `erasable` — syntax that leaves no runtime code
- `tsc --noEmit` — the actual type check
- `tsx` — fallback for full TypeScript

CommonJS Modules

COMMONJS

CommonJS (CJS) is Node's original module system. A file exports by assigning to `module.exports` and imports with `require()`, which loads **synchronously** and returns whatever the other file exported. Each module runs once; later `require()` calls return the cached result. Most older packages and many configs still use CJS, so you need to read it even if you write ES modules.

NOTES

- A file is CJS if it ends in `.cjs`, or is `.js` without `"type": "module"`
- `require` can be called anywhere, even inside `if`
- Extension is optional: `.js`, `.json`, `.node` are tried
- Code is wrapped in a function, so top-level `return` works

EXPORTING & REQUIRING

```
// math.js
const PI = 3.14159;
function area(r) { return PI * r * r; }
module.exports = { area, PI };            // export an object

// app.js
const { area } = require('./math');       // relative file, extension optional
const fs = require('node:fs');            // core module
const _ = require('lodash');              // package from node_modules
console.log(area(2));                     // 12.56636

exports.hello = () => 'hi';               // ✔ adds to module.exports
exports = { hello: () => 'hi' };          // ✖ rebinds local var only

require.resolve('lodash');                // '/app/node_modules/lodash/lodash.js'
delete require.cache[require.resolve('./math')];  // force reload
if (require.main === module) main();      // true only when run directly
```

CODE EXPLAINED

- → `module.exports` — the value `require` returns
- → `exports` — a shortcut that points at `module.exports`
- → `require.resolve` — path only, does not load
- → `require.cache` — keyed by absolute path
- → `require.main` — the entry module

HOW REQUIRE() RESOLVES

SPECIFIER	LOOKS IN
`'node:fs'`	Core module
`'./x'`	x.js, x.json, x/index.js
`'/abs/x'`	Absolute path
`'pkg'`	node_modules, walking up
`'pkg/sub'`	"exports" map, else file

MODULE WRAPPER VARIABLES

NAME	VALUE
`exports`	Alias of module.exports
`require`	Loader for this file
`module`	This module's record
`__filename`	Absolute file path
`__dirname`	Folder of this file

EXPORT PATTERNS

PATTERN	IMPORT WITH
`module.exports = fn`	const fn = require()
`module.exports = {a, b}`	const {a} = require()
`exports.a = 1`	require().a
`module.exports = class`	new (require())()

COMMON MISTAKES

- Assigning `exports = …`; only `module.exports = …` replaces the export
- Circular requires: one side gets a half-filled object; move shared code to a third module
- Mutating an object from `require`; every importer shares the same cached instance

TIPS

- Write new code as ESM; use CJS when a tool or config file requires it
- Use `.cjs` for a CJS file inside a `"type": "module"` package
- Lazy-require heavy modules inside the function that needs them

SUMMARY

- `module.exports` — what a file exports
- `require()` — synchronous, cached import
- `require.resolve` — find a module's path
- `require.main` — detect direct execution

ES Modules

ES MODULES

ES modules (ESM) are the JavaScript standard module system, shared with browsers. Imports are **static**: Node reads every `import` before running any code, links the files, then executes them. That enables **top-level await** and live bindings. Node treats a file as ESM if it ends in `.mjs`, or is `.js` inside a package whose `package.json` has `"type": "module"`.

NOTES

- Relative imports need the full extension: `'./math.js'`
- ESM always runs in strict mode
- A `.js` file with no `"type"` is parsed as ESM if it uses `import` / `export` syntax
- Imports are hoisted: they run before the file body

IMPORT & EXPORT FORMS

```
// math.js  (package.json has "type": "module")
export const PI = 3.14159;                          // named export
export function area(r) { return PI * r * r; }
export default class Circle {}                      // default export

// app.js
import Circle, { area, PI as pi } from './math.js'; // default, named, alias
import * as math from './math.js';                  // namespace: math.area(2)
import { readFile } from 'node:fs/promises';
import config from './config.json' with { type: 'json' };  // JSON

const text = await readFile('notes.txt', 'utf8');   // top-level await
const { default: chalk } = await import('chalk');   // dynamic import
export { area as circleArea } from './math.js';     // re-export under a new name
export * from './shapes.js';                        // re-export everything named
```

CODE EXPLAINED

- → `export default` — one per module, import with any name
- → `as` — renames on import or export
- → `* as` — all named exports as one object
- → `with { type: 'json' }` — required for JSON imports
- → `import()` — returns a promise, works in CJS too
- → `export *` — does not include the default export

ESM VS COMMONJS

FEATURE	ESM	CJS
Syntax	import / export	require / exports
Loading	Static, async	Dynamic, sync
Top-level await	✔	✖
File extension	Required	Optional
__dirname	import.meta.dirname	Built in
Top-level this	undefined	module.exports
Bindings	Live	Copied values

HOW NODE PICKS THE FORMAT

FILE	TREATED AS
.mjs	ESM, always
.cjs	CommonJS, always
.js + "type": "module"	ESM
.js + "type": "commonjs"	CommonJS
.js, no "type"	Detected from syntax
--input-type=module	ESM for -e / stdin

COMMON MISTAKES

- Leaving off `.js` in a relative import gives `ERR_MODULE_NOT_FOUND`
- Mixing `require` into an ES module; it's not defined there (see Sheet 08)
- Expecting `export *` to forward the default; re-export it by name

TIPS

- Set `"type"` explicitly; syntax detection costs a parse and prints a warning
- Prefer named exports; they autocomplete and rename safely
- Use `import()` to load optional or heavy modules only when needed

SUMMARY

- `export` — named or default
- `import` — static, hoisted, needs extensions
- `import()` — dynamic, returns a promise
- `"type": "module"` — makes `.js` files ESM

CJS ↔ ESM Interop

MIXING THE TWO SYSTEMS

Most projects end up with both formats: ESM app code that uses CJS packages, or CJS code that needs an ESM-only package. **ESM can always import CJS.** CJS can `require()` an ES module directly as long as that module (and everything it imports) has **no top-level await**. For anything else, use `await import()`. Inside ESM, `createRequire()` gives you a working `require`.

NOTES

- `require(esm)` works without a flag since Node 22.12 / 20.19
- Named imports from CJS depend on static analysis of the CJS file
- The CJS `module.exports` object is always the ESM default import

CROSSING THE BOUNDARY

```
// ── ESM importing CommonJS ──────────────────────
import legacy from './legacy.cjs';          // default = module.exports
import { helper } from './legacy.cjs';      // named, if Node can detect it
import pkg from 'old-cjs-package';
const { helper: h } = pkg;                  // fallback when named import fails

// ── CommonJS requiring ESM ──────────────────────
const { area } = require('./math.mjs');     // namespace object
const Circle = require('./math.mjs').default;   // default export
require('./uses-await.mjs');                // ✖ ERR_REQUIRE_ASYNC_MODULE
async function load() {
  const { default: chalk } = await import('chalk');  // always works
}

// ── require() inside an ES module ───────────────
import { createRequire } from 'node:module';
const require = createRequire(import.meta.url);
const data = require('./data.json');        // sync JSON load
```

CODE EXPLAINED

- → `import legacy` — gets the whole `module.exports`
- → `{ helper }` — works for plain `exports.x =` patterns
- → `.default` — where an ESM default export lands
- → `ERR_REQUIRE_ASYNC_MODULE` — top-level await can't be loaded synchronously
- → `createRequire` — needs a file URL or path as its base

INTEROP ERRORS

ERROR	FIX
ERR_REQUIRE_ASYNC_MODULE	Use await import()
ERR_REQUIRE_ESM	Old Node: upgrade or import()
ERR_MODULE_NOT_FOUND	Add the file extension
does not provide an export named	Import default, destructure
require is not defined	createRequire()
__dirname is not defined	import.meta.dirname

DUAL PACKAGE (EXPORTS)

```
"exports": {
  ".": {
    "import": "./dist/index.mjs",
    "require": "./dist/index.cjs"
  }
}
```

COMMON MISTAKES

- Shipping a dual package with module-level state; ESM and CJS copies don't share it (the dual package hazard)
- Assuming every CJS named export is importable; computed exports can't be detected
- Adding top-level await to a library breaks all of its CJS users

TIPS

- On Node 22.12+, you can usually drop the CJS build and ship ESM only
- Keep library entry points free of top-level await so `require()` works
- Check a package's format: look at its `"type"` and `"exports"`

SUMMARY

- `ESM → CJS` — import works; default is `module.exports`
- `CJS → ESM` — `require()` if no top-level await
- `import()` — works from either side
- `createRequire` — `require` inside ESM

package.json

THE PROJECT MANIFEST

`package.json` describes a project: its name and version, its dependencies, the scripts you run, and how other code may import it. Node itself reads only a few fields (`"type"`, `"exports"`, `"imports"`, `"main"`); npm and other tools read the rest. Create one with `npm init -y`.

NOTES

- Must be valid JSON: no comments, no trailing commas
- `"exports"` wins over `"main"` and blocks unlisted paths
- `"private": true` prevents accidental `npm publish`
- Set fields from the CLI: `npm pkg set type=module`

A TYPICAL PACKAGE.JSON

```
{
  "name": "@acme/notes-api",
  "version": "1.4.0",
  "private": true,
  "type": "module",
  "exports": { ".": "./src/index.js", "./client": "./src/client.js" },
  "imports": { "#db/*": "./src/db/*.js" },
  "bin": { "notes": "./bin/cli.js" },
  "scripts": {
    "dev": "node --watch --env-file=.env src/server.js",
    "start": "node src/server.js",
    "test": "node --test"
  },
  "engines": { "node": ">=24" },
  "dependencies": { "fastify": "^5.2.0" },
  "devDependencies": { "typescript": "~5.9.0" }
}
```

CODE EXPLAINED

- → `@acme/` — a scoped package name
- → `exports` — public entry points; `import '@acme/notes-api/client'`
- → `imports` — private aliases: `import db from '#db/users'`
- → `bin` — commands installed on PATH
- → `engines` — supported Node range
- → `devDependencies` — not installed with `--omit=dev`

DEPENDENCY FIELDS

FIELD	USE FOR
`dependencies`	Needed at runtime
`devDependencies`	Build, test, lint only
`peerDependencies`	Host must provide (plugins)
`optionalDependencies`	Install may fail safely
`overrides`	Force a nested version

EXPORTS CONDITIONS

KEY	MATCHES
`"types"`	TypeScript (list first)
`"import"`	import / import()
`"require"`	require()
`"node"`	Any Node environment
`"default"`	Fallback (list last)

SPECIAL SCRIPTS

SCRIPT	RUNS
`start / test`	npm start / npm test
`pre<name>`	Before <name>
`post<name>`	After <name>
`prepare`	After install, before pack
`postinstall`	After package installs

COMMON MISTAKES

- Adding `"exports"` without listing subpaths users import; they get `ERR_PACKAGE_PATH_NOT_EXPORTED`
- Putting runtime packages in `devDependencies`; production installs omit them
- Editing versions by hand and not updating the lockfile; run `npm install` after

TIPS

- Use `"imports"` `#aliases` instead of long `../../` paths
- Read fields from code with `import pkg from './package.json' with { type: 'json' }`
- Expose `"./package.json"` in `"exports"` for tools that read it

SUMMARY

- `type` — ESM or CJS for `.js`
- `exports` — public entry points
- `imports` — internal `#` aliases
- `scripts` — named project commands

npm CLI

MANAGING DEPENDENCIES

npm ships with Node. It installs packages into `node_modules`, records the version ranges you asked for in `package.json`, and records the **exact** resolved versions in `package-lock.json`. Commit the lockfile: it's what makes every install on every machine produce the same tree. **npx** runs a package's command without installing it globally.

NOTES

- Short forms: `i` install · `rm` uninstall · `-D` dev · `-E` exact
- Never commit `node_modules`; add it to `.gitignore`
- Project settings live in `.npmrc`

EVERYDAY COMMANDS

```
npm init -y                  # create package.json with defaults
npm install                  # install everything in package.json
npm install fastify          # add a runtime dependency
npm install -D vitest        # add a dev dependency
npm install zod@3.24.1 -E    # exact version, no ^ range
npm uninstall lodash         # remove a dependency
npm ci                       # clean install from the lockfile (CI, Docker)
npm ci --omit=dev            # production install
npm outdated                 # what has newer versions
npm update                   # upgrade within the allowed ranges
npm audit                    # known vulnerabilities
npm ls fastify               # where a package sits in the tree
npm run build                # run a script (npm run lists them all)
npx prettier --write .       # run a package's binary
npm view fastify version     # latest published version
```

CODE EXPLAINED

→ `install` — may update the lockfile

→ `ci` — deletes `node_modules`, never changes the lockfile, fails if it's out of sync

→ `update` — won't cross a major version on `^`

→ `npx` — uses the local copy if installed, else downloads

→ `ls` — explains why a package is present

SEMVER RANGES

RANGE	ALLOWS
`^1.4.2`	1.4.2 to <2.0.0
`~1.4.2`	1.4.2 to <1.5.0
`1.4.2`	Exactly 1.4.2
`>=1.4 <2`	Explicit range
`^0.3.1`	0.3.1 to <0.4.0 (0.x is strict)
`*`	Anything (avoid)

VERSIONING YOUR PACKAGE

COMMAND	1.4.2 →
`npm version patch`	1.4.3
`npm version minor`	1.5.0
`npm version major`	2.0.0
`npm version prerelease`	1.4.3-0
`npm publish`	Upload to registry

USEFUL .NPMRC SETTINGS

SETTING	EFFECT
`save-exact=true`	No ^ on install
`engine-strict=true`	Enforce engines
`ignore-scripts=true`	Skip install scripts
`registry=URL`	Private registry

COMMON MISTAKES

- Running `npm install` in CI; it can change the lockfile, so use `npm ci`
- Deleting `package-lock.json` to fix errors; it silently upgrades everything
- Running `npm audit fix --force` blindly; it can jump major versions

TIPS

- Check a package before adding it: `npm view <pkg>` and its repo activity
- Use `npm pkg get version` in scripts instead of parsing JSON
- Pass args to a script after `--`: `npm test -- --watch`

SUMMARY

- `npm install` — add packages, update lockfile
- `npm ci` — exact, reproducible install
- `^ / ~` — major / minor ranges
- `npx` — run a package's command

pnpm, Yarn & Workspaces

ALTERNATIVES & MONOREPOS

pnpm and **Yarn** are drop-in replacements for npm. pnpm stores each package version once on disk and links it into projects, so installs are fast and `node_modules` is strict (you can only import what you declared). **Workspaces** let one repo hold several packages that depend on each other; the package manager links them together instead of downloading them.

NOTES

- Pick one manager per repo; each writes its own lockfile
- Pin it with `"packageManager": "pnpm@10.x.x"`
- Corepack is not bundled from Node 25 on; install it with `npm i -g corepack`
- Internal packages use the `workspace:*` protocol (pnpm, Yarn)

WORKSPACE SETUP

```
// package.json at the repo root (npm, Yarn)
{ "private": true, "workspaces": ["packages/*", "apps/*"] }

// apps/web/package.json — depends on a sibling package
{ "dependencies": { "@acme/ui": "workspace:*" } }
```

```
# pnpm-workspace.yaml (pnpm uses this instead of "workspaces")
packages:
  - "packages/*"
  - "apps/*"
```

```
npm install -w apps/web zod          # add a dep to one workspace
npm run build --workspaces           # run in every workspace
pnpm --filter web dev                # run "dev" in one package
pnpm -r build                        # run in all packages, in dependency order
```

CODE EXPLAINED

- → `private: true` — required at the root
- → `workspace:*` — link the local copy; replaced with a real version on publish
- → `-w` — target one workspace (npm)
- → `--filter` — target by package name (pnpm)
- → `-r` — recursive across all packages

COMMAND EQUIVALENTS

NPM	PNPM	YARN
`npm install`	`pnpm install`	`yarn`
`npm i pkg`	`pnpm add pkg`	`yarn add pkg`
`npm i -D pkg`	`pnpm add -D pkg`	`yarn add -D pkg`
`npm rm pkg`	`pnpm remove pkg`	`yarn remove pkg`
`npm ci`	`pnpm i --frozen-lockfile`	`yarn --immutable`
`npm run dev`	`pnpm dev`	`yarn dev`
`npx tool`	`pnpm dlx tool`	`yarn dlx tool`

LOCKFILES & LINKING

ITEM	PURPOSE
`package-lock.json`	npm lockfile
`pnpm-lock.yaml`	pnpm lockfile
`yarn.lock`	Yarn lockfile
`npm link`	Use a local package globally
`overrides`	npm / pnpm version pinning
`resolutions`	Yarn version pinning

COMMON MISTAKES

- Committing two lockfiles; tools disagree about which versions are installed
- Importing an undeclared package under pnpm; it worked in npm only because of hoisting
- Forgetting to build a workspace package before the app that imports it

TIPS

- Use pnpm for monorepos; strictness catches missing dependencies early
- Add a task runner (Turborepo, Nx) once builds get slow
- Prefer workspaces over `npm link`; links break across Node versions

SUMMARY

- `pnpm / yarn` — npm alternatives
- `workspaces` — many packages, one repo
- `workspace:*` — link a sibling package
- `packageManager` — pins the tool

The Event Loop

HOW NODE SCHEDULES WORK

The **event loop** is what lets one thread juggle many tasks. Node runs your synchronous code to the end, then loops through **phases**, each with its own queue of callbacks: expired timers, finished I/O, `setImmediate`, and close events. Between every callback it drains two higher-priority queues: `process.nextTick` first, then **microtasks** (promise reactions). The loop exits when nothing is left to wait for.

NOTES

- Only one callback runs at a time; there are no data races in JS code
- A callback that takes 200 ms delays everything else by 200 ms
- Timer delays are minimums, never guarantees

EXECUTION ORDER

```
// order.cjs — run with: node order.cjs
const fs = require('node:fs');

console.log('1 sync');
setTimeout(() => console.log('timeout'), 0);
setImmediate(() => console.log('immediate'));
Promise.resolve().then(() => console.log('4 promise'));
process.nextTick(() => console.log('3 nextTick'));
console.log('2 sync');
// prints 1, 2, 3, 4, then timeout + immediate in either order

fs.readFile(__filename, () => {            // inside an I/O callback…
  setTimeout(() => console.log('B timeout'), 0);
  setImmediate(() => console.log('A immediate'));   // …immediate always runs first
});

const end = Date.now() + 500;
while (Date.now() < end) {}              // blocks: every timer above waits 500 ms
```

CODE EXPLAINED

- → `sync` — all top-level code runs before any callback
- → `nextTick` — runs as soon as the current operation ends
- → `then` — microtask, right after the nextTick queue
- → `timeout vs immediate` — order from the main module depends on process timing
- → After I/O, the check phase comes before timers
- → `while` — busy loop starves the event loop

LOOP PHASES (IN ORDER)

PHASE	RUNS
`timers`	Expired setTimeout / setInterval
`pending`	Deferred system callbacks
`poll`	I/O callbacks; waits for I/O
`check`	setImmediate callbacks
`close`	'close' events (sockets)

PRIORITY QUEUES

QUEUE	FILLED BY
`nextTick`	process.nextTick()
`microtask`	.then, await, queueMicrotask
`drained`	Fully, after every callback
`order`	nextTick, then microtasks

WHAT BLOCKS THE LOOP

CAUSE	FIX
`*Sync fs calls`	fs/promises
`Huge JSON.parse`	Stream or worker
`Bad regex`	Bound input size
`Hashing in a loop`	Async crypto API
`Heavy math`	worker_threads

COMMON MISTAKES

- Expecting `setTimeout(fn, 0)` to run before a promise; microtasks always win
- Recursive `process.nextTick` starves I/O completely; use `setImmediate`
- In ES modules, the promise can log before `nextTick`; top-level ESM code is itself a microtask

TIPS

- Break long loops into chunks with `await setImmediate()` from `timers/promises`
- Measure loop lag with `monitorEventLoopDelay()` (Sheet 53)
- Prefer `queueMicrotask` over `nextTick` in new code

SUMMARY

- `sync code` — always finishes first
- `nextTick → microtasks` — after each callback
- `timers → poll → check` — phase order
- `blocking` — delays every pending callback

Timers

SCHEDULING CODE LATER

Node has the browser's `setTimeout` and `setInterval`, plus `setImmediate` (run after the current I/O phase). In Node they return a **Timeout object**, not a number. The `node:timers/promises` module offers **promise versions** you can `await`, cancel with an `AbortSignal`, and loop over with `for await`.

NOTES

- Delays are clamped to at least 1 ms and max ~24.8 days
- An active timer keeps the process alive unless you `unref()` it
- Extra arguments after the delay are passed to the callback

CALLBACK & PROMISE TIMERS

```
import { setTimeout as sleep, setInterval as every }
  from 'node:timers/promises';

const t = setTimeout((name) => console.log(`hi ${name}`), 1000, 'Ada'); // hi Ada
clearTimeout(t);                          // cancel before it fires

const id = setInterval(() => console.log('tick'), 500);
setTimeout(() => clearInterval(id), 2100); // tick ×4, then stops

setTimeout(flushLogs, 60_000).unref();    // won't keep the process alive

await sleep(250);                         // pause 250 ms
const v = await sleep(100, 'done');       // resolves with 'done'

const ac = new AbortController();
sleep(5000, null, { signal: ac.signal })
  .catch(e => console.log(e.name));      // AbortError
ac.abort();

for await (const _ of every(1000)) {      // async interval
  if (await poll()) break;                // break clears the interval
}
```

CODE EXPLAINED

- → `as sleep` — rename to avoid shadowing the global
- → `'Ada'` — extra arg forwarded to the callback
- → `unref()` — background timer, exit allowed
- → `sleep(ms, value)` — resolves with value
- → `signal` — abort rejects with `AbortError`
- → `for await` — interval that can't overlap itself

TIMER FUNCTIONS

FUNCTION	CANCEL WITH
`setTimeout(fn, ms)`	clearTimeout(t)
`setInterval(fn, ms)`	clearInterval(t)
`setImmediate(fn)`	clearImmediate(t)
`queueMicrotask(fn)`	Can't cancel

TIMEOUT OBJECT

METHOD	EFFECT
`unref()`	Don't hold process open
`ref()`	Undo unref()
`hasRef()`	true / false
`refresh()`	Restart the countdown
`[Symbol.dispose]`	Clear via using

TIMEOUTS ON OTHER WORK

API	USE
`AbortSignal.timeout(ms)`	Auto-abort signal
`AbortSignal.any([...])`	Combine signals
`fetch(url, { signal })`	Request timeout

COMMON MISTAKES

- `setInterval` with async work: runs overlap if one takes longer than the delay
- Storing the timer in a number type (TS); use `ReturnType<typeof setTimeout>`
- A forgotten interval keeps the process from ever exiting

TIPS

- Debounce with `t.refresh()` instead of clearing and recreating
- Use `AbortSignal.timeout()` rather than racing promises against a timer
- Mock timers in tests with `mock.timers` (Sheet 49)

SUMMARY

- `setTimeout` — run once after a delay
- `setInterval` — run repeatedly
- `timers/promises` — awaitable timers
- `unref()` — let the process exit

Callbacks & Error-First Style

(err, data) => {} · util.promisify() · util.callbackify() · callback hell · returning early on err

THE ORIGINAL ASYNC STYLE

Before promises, every async Node API took a **callback** as its last argument and called it with `(err, result)`: the **error-first** convention. If `err` is non-null, the operation failed and `result` is meaningless. Most core modules now have promise versions, but you still meet callbacks in older packages, event handlers, and some core APIs. `util.promisify` converts them.

NOTES

- Callbacks don't return values; results only flow into the callback
- `try/catch` around a callback API doesn't catch its async errors
- A callback must be called exactly once

ERROR-FIRST CALLBACKS & CONVERSION

```
import fs from 'node:fs';
import { promisify, callbackify } from 'node:util';

fs.readFile('config.json', 'utf8', (err, text) => {
  if (err) return console.error(err.code);   // 'ENOENT' — return early!
  console.log(JSON.parse(text).port);        // 3000
});

function getUser(id, cb) {                     // writing your own
  if (!id) return cb(new Error('id required'));
  setImmediate(() => cb(null, { id, name: 'Ada' }));  // always async
}

const getUserAsync = promisify(getUser);       // callback → promise
const user = await getUserAsync(7);            // { id: 7, name: 'Ada' }

const legacyApi = callbackify(async (id) => ({ id }));  // promise → callback
legacyApi(7, (err, u) => console.log(u));      // { id: 7 }
```

CODE EXPLAINED

- → `return console.error` — stops the success path running too
- → `cb(null, value)` — `null` means no error
- → `setImmediate` — never call a callback synchronously some of the time
- → `promisify` — expects `(…args, cb)` with `(err, value)`
- → `callbackify` — for APIs that still require callbacks

CALLBACK → PROMISE EQUIVALENTS

CALLBACK	PROMISE
`fs.readFile(p, cb)`	`fsp.readFile(p)`
`dns.lookup(h, cb)`	`dnsp.lookup(h)`
`setTimeout(cb, ms)`	`timersp.setTimeout(ms)`
`stream.pipeline(…, cb)`	`streamp.pipeline(…)`
`crypto.scrypt(…, cb)`	`promisify(crypto.scrypt)`
`emitter.once('x', cb)`	`events.once(e, 'x')`

CALLBACK RULES

RULE	WHY
`err first`	promisify depends on it
`call once`	Twice causes double replies
`always async`	Avoids "Zalgo" ordering bugs
`return after err`	Stops fall-through
`Error objects`	Keep stack traces

COMMON MISTAKES

- Forgetting `return` after handling `err`; the success code runs with `undefined`
- Throwing inside a callback; nothing up the stack can catch it, so the process crashes
- Promisifying a method loses `this`; use `promisify(obj.fn).bind(obj)`

TIPS

- Check for a `/promises` module before reaching for `promisify`
- Flatten nested callbacks ("callback hell") by converting to `async` / `await`
- Pass `Error` objects, never strings, as the first argument

SUMMARY

- `(err, result)` — error-first convention
- `return cb(err)` — handle and exit
- `promisify` — callback API → promise
- `callbackify` — promise → callback API

Promise-Based Core APIs

PROMISE VERSIONS OF CORE MODULES

Most core modules expose a **promise API** under a `/promises` subpath, so you can write plain `async/await` code with `try/catch`. Combine them with the standard **Promise combinators** to run work in parallel. Since Node 15, an **unhandled rejection crashes the process**, the same as an uncaught exception.

NOTES

- Import named functions: `import { readFile } from 'node:fs/promises'`
- Rejections carry the same `err.code` values as callbacks
- `events.once()` turns an event into a promise

ASYNC / AWAIT WITH CORE APIS

```
import { readFile, writeFile } from 'node:fs/promises';
import { pipeline } from 'node:stream/promises';
import { once } from 'node:events';

try {
  const cfg = JSON.parse(await readFile('config.json', 'utf8'));
  await writeFile('out.txt', `port=${cfg.port}\n`);
} catch (err) {
  if (err.code === 'ENOENT') console.log('no config yet');  // expected
  else throw err;                                           // unexpected
}

const [a, b] = await Promise.all([readFile('a.txt'), readFile('b.txt')]);
const results = await Promise.allSettled(urls.map(u => fetch(u)));
results.filter(r => r.status === 'rejected').length;       // failure count

await pipeline(src, gzip, dest);          // resolves when the stream finishes
const [req] = await once(server, 'request');   // wait for one event
```

CODE EXPLAINED

- → `try/catch` — catches rejected awaits
- → `err.code` — branch on the error type, rethrow the rest
- → `Promise.all` — parallel, fails fast on the first rejection
- → `allSettled` — waits for all, never rejects
- → `once()` — resolves with the event's args as an array

PROMISE MODULES

IMPORT	COVERS
`node:fs/promises`	Files, dirs, FileHandle
`node:stream/promises`	pipeline, finished
`node:timers/promises`	setTimeout, setInterval
`node:dns/promises`	lookup, resolve
`node:readline/promises`	question()
`node:events`	once(), on()

COMBINATORS

METHOD	RESOLVES	REJECTS
`all`	All values	First failure
`allSettled`	All outcomes	Never
`race`	First to settle	First to settle
`any`	First success	All failed

--UNHANDLED-REJECTIONS

MODE	BEHAVIOR
`throw`	Crash (default)
`strict`	Throw before the event fires
`warn`	Print warning only
`none`	Silence (avoid)

COMMON MISTAKES

- Calling an async function without `await` or `.catch()`; its error becomes an unhandled rejection
- Awaiting in a loop when the calls are independent; use `Promise.all`
- Swallowing every error in `catch`; rethrow what you didn't expect

TIPS

- Use `allSettled` for batch jobs where partial success is fine
- Mark intentional background promises: `void task().catch(log)`
- Lint with `no-floating-promises` (typescript-eslint)

SUMMARY

- `*/promises` — awaitable core modules
- `Promise.all` — parallel, fail fast
- `allSettled` — parallel, collect all
- `unhandled rejection` — crashes by default

Async Patterns & Cancellation

sequential vs parallel await · for await…of · AbortController · signal option · concurrency limits · events.on()

CONTROLLING ASYNC WORK

Real programs need more than `await`: run independent work **in parallel**, cap how many tasks run **at once**, process data that arrives over time with **async iteration**, and **cancel** work that's no longer needed. Node standardizes cancellation on `AbortController`: most async core APIs and `fetch` accept a `signal` option.

NOTES

- Aborting rejects with an `AbortError` (`err.name`)
- A signal can only be aborted once
- Starting a promise starts the work; `await` only waits for it

PATTERNS

```
const a = await getA(); const b = await getB();    // sequential: ~2s
const [a2, b2] = await Promise.all([getA(), getB()]); // parallel: ~1s

// Limit concurrency: at most 5 downloads at a time
async function mapLimit(items, limit, fn) {
  const out = [], queue = items.entries();
  const worker = async () => {
    for (const [i, x] of queue) out[i] = await fn(x);
  };
  await Promise.all(Array.from({ length: limit }, worker));
  return out;
}
await mapLimit(urls, 5, url => fetch(url).then(r => r.json()));

// Cancellation: user timeout OR manual abort
const ac = new AbortController();
const signal = AbortSignal.any([ac.signal, AbortSignal.timeout(3000)]);
const res = await fetch(url, { signal });       // rejects if either fires

for await (const line of rl) { … }              // consume data as it arrives
const { promise, resolve } = Promise.withResolvers();   // resolve from outside
```

CODE EXPLAINED

- → `Promise.all` — both start immediately
- → `items.entries()` — shared iterator; each worker pulls the next item
- → `out[i]` — keeps results in input order
- → `AbortSignal.any` — aborts when any input aborts
- → `for await` — works on streams, readline, `events.on()`
- → `withResolvers` — wrap event-style APIs

CHOOSING A PATTERN

SITUATION	USE
`B needs A's result`	Sequential await
`Few independent tasks`	Promise.all
`Hundreds of tasks`	Concurrency limit (p-limit)
`Data arrives over time`	for await…of
`Need a deadline`	AbortSignal.timeout
`First response wins`	Promise.any + abort rest

ACCEPTING A SIGNAL IN YOUR CODE

API	USE
`signal.aborted`	Check before each step
`signal.throwIfAborted()`	Bail out with AbortError
`signal.reason`	Why it was aborted
`addEventListener('abort')`	Clean up resources

COMMON MISTAKES

- `Promise.all` over 10,000 URLs; you open 10,000 sockets at once
- `arr.forEach(async x => …)` ; nothing waits for those promises
- Racing a timeout with `Promise.race` leaves the losing request running

TIPS

- Accept `{ signal }` in your own async functions and pass it down
- Use `p-limit` or `p-map` rather than hand-rolling pools in apps
- Abort leftover requests after `Promise.any` resolves

SUMMARY

- `Promise.all` — run in parallel
- `limit` — cap concurrent work
- `for await` — iterate async sources
- `AbortController` — cancel work

EventEmitter

EVENTS IN NODE

`EventEmitter` is Node's publish/subscribe building block. Objects **emit** named events and listeners **subscribe** with `on()`. Servers, streams, sockets, child processes, and `process` itself are all emitters. Listeners run **synchronously**, in the order they were added, at the moment `emit()` is called.

NOTES

- An `'error'` event with no listener **throws** and usually crashes
- More than 10 listeners on one event prints a leak warning
- `emit()` returns `true` if anyone was listening

EMITTING & LISTENING

```
import { EventEmitter, once, on } from 'node:events';

class Order extends EventEmitter {
  pay(amount) {
    if (amount <= 0) return this.emit('error', new Error('bad amount'));
    this.emit('paid', { amount, at: Date.now() });
  }
}

const order = new Order();
const log = (e) => console.log('paid', e.amount);
order.on('paid', log);                      // every time
order.once('paid', () => console.log('first payment only'));
order.on('error', (err) => console.error(err.message));   // always add one
order.pay(25);                              // paid 25 / first payment only
order.off('paid', log);                     // remove (same function reference)

const [evt] = await once(order, 'paid');    // promise for the next event
for await (const [e] of on(order, 'paid')) { … }   // async stream of events
```

CODE EXPLAINED

- → `extends EventEmitter` — give your class events
- → `emit(name, …args)` — args go to every listener
- → `once` — auto-removed after one call
- → `'error'` listener — prevents a crash
- → `off` — needs the exact function you added
- → `events.once` — also rejects if `'error'` fires first

EMITTER METHODS

METHOD	DOES
`on / addListener`	Subscribe
`once`	Subscribe for one call
`prependListener`	Run before others
`off / removeListener`	Unsubscribe one
`removeAllListeners`	Unsubscribe all
`listenerCount`	How many listeners
`setMaxListeners(n)`	Raise warning limit

NODE:EVENTS HELPERS

HELPER	DOES
`once(em, name)`	Promise of next event
`on(em, name)`	Async iterator of events
`{ signal }`	Cancel either helper
`captureRejections`	Async listener errors → 'error'
`EventTarget`	Web-standard alternative

BUILT-IN EMITTERS

OBJECT	KEY EVENTS
`http.Server`	request, listening
`Readable`	data, end, error
`net.Socket`	data, close
`ChildProcess`	exit, close
`process`	exit, SIGINT

COMMON MISTAKES

- Adding listeners per request and never removing them; memory leak plus the 10-listener warning
- `off()` with an inline arrow function; it's a new function, so nothing is removed
- Async listeners that throw become unhandled rejections unless `captureRejections` is on

TIPS

- Emit data objects, not long positional argument lists
- Don't silence the leak warning; find the listener you forgot to remove
- Use `events.once(server, 'listening')` in tests and startup code

SUMMARY

- `on / emit` — subscribe and publish
- `once` — fire one time
- `'error'` — always listen for it
- `events.once/on` — promise-based events

The process Object

YOUR PROGRAM'S RUNTIME HANDLE

The global `process` object connects your code to the running Node process and the OS around it. It holds the **command-line arguments**, **environment variables**, working directory, platform info, and memory stats, and it controls how and when the process **exits**. It's also an EventEmitter for signals and lifecycle events (Sheet 19).

NOTES

- Always available; no import needed (or `import process from 'node:process'`)
- Every `process.env` value is a string or `undefined`
- `cwd()` is where node was launched, not where the file lives

READING PROCESS STATE

```
// node app.js serve --port 8080
process.argv;                // ['/usr/bin/node', '/app/app.js', 'serve', …]
process.argv.slice(2);       // ['serve', '--port', '8080']
process.env.HOME;            // '/home/ada'
process.env.PORT ?? '3000'; // env values are strings
process.cwd();               // '/app'
process.chdir('/tmp');       // change the working directory
process.pid;                 // 48213
process.platform;            // 'linux' | 'darwin' | 'win32'
process.arch;                // 'x64' | 'arm64'
process.version;             // 'v26.1.0'
process.versions.v8;         // '14.6.…'
process.uptime();            // 12.03 (seconds)
process.memoryUsage().heapUsed;  // 5832144 (bytes)
process.exitCode = 1;        // exit with 1 when the loop empties
process.exit(2);             // exit now; pending async work is dropped
```

CODE EXPLAINED

- → `argv[0]` — node binary; `argv[1]` — script path
- → `slice(2)` — just the user's arguments
- → `??` — fallback when the variable is unset
- → `exitCode` — graceful: lets writes and logs finish
- → `exit()` — immediate; use only when you must stop now

PROPERTIES

PROPERTY	VALUE
`argv`	Command-line args
`env`	Environment variables
`execArgv`	Node flags, e.g. --watch
`platform / arch`	OS and CPU
`pid / ppid`	Process IDs
`stdin / stdout / stderr`	Standard streams
`exitCode`	Exit code to use later

METHODS

METHOD	DOES
`cwd()`	Working directory
`chdir(dir)`	Change it
`exit(code)`	Exit immediately
`memoryUsage()`	rss, heap, external
`cpuUsage()`	user / system µs
`hrtime.bigint()`	Nanosecond timer
`kill(pid, sig)`	Send a signal

COMMON MISTAKES

- Treating `process.env.DEBUG === true` as boolean; it's the string `'true'`
- Calling `process.exit()` right after `stdout.write()`; output can be cut off when piped
- Reading files relative to `cwd()`; it changes depending on where the user runs the command

TIPS

- Parse and validate env vars once at startup (Sheet 20)
- Use `process.hrtime.bigint()` or `performance.now()` for timing, not `Date.now()`
- Prefer setting `exitCode` and letting the process end on its own

SUMMARY

- `process.argv` — command-line arguments
- `process.env` — string environment variables
- `process.cwd()` — launch directory
- `process.exitCode` — graceful exit status

Signals & Graceful Shutdown

SHUTTING DOWN CLEANLY

Servers get stopped by **signals**: `SIGTERM` from Docker, Kubernetes, and systemd, `SIGINT` from Ctrl+C. By default Node exits immediately, dropping in-flight requests and unflushed data. A **graceful shutdown** handler stops accepting new work, lets current work finish, closes resources, and then exits, with a hard deadline in case something hangs.

NOTES

- `SIGKILL` and `SIGSTOP` can't be caught
- Docker sends `SIGKILL` 10 s after `SIGTERM` by default
- Windows doesn't send `SIGTERM`; `SIGINT` works
- The `'exit'` handler can only run synchronous code

GRACEFUL SHUTDOWN HANDLER

```
import { createServer } from 'node:http';
const server = createServer(handler).listen(3000);

let stopping = false;
async function shutdown(signal) {
  if (stopping) return;                  // ignore a second Ctrl+C
  stopping = true;
  console.log(`${signal}: draining`);
  setTimeout(() => process.exit(1), 10_000).unref();   // hard deadline
  server.close();                        // stop accepting; finish in-flight
  server.closeIdleConnections();         // drop idle keep-alive sockets
  await db.end();                        // close pools, flush logs
  process.exitCode = 0;                  // exit once the loop is empty
}
process.on('SIGTERM', shutdown);         // docker stop, k8s, systemd
process.on('SIGINT', shutdown);          // Ctrl+C

process.on('exit', (code) => console.log('exit', code));  // sync only
```

CODE EXPLAINED

- `stopping` flag — handlers can fire more than once
- `unref()` — the deadline itself won't keep the process alive
- `server.close()` — waits for open requests to finish
- `closeIdleConnections` — otherwise keep-alive sockets delay close
- No `process.exit(0)` — exit happens naturally when work is done

SIGNALS

SIGNAL	SENT BY
SIGINT	Ctrl+C in a terminal
SIGTERM	docker stop, kill, k8s
SIGHUP	Terminal closed
SIGUSR2	Free for your own use
SIGUSR1	Reserved: starts debugger
SIGKILL	kill -9 (uncatchable)

EXIT EVENTS

EVENT	WHEN
beforeExit	Loop empty; can add work
exit	Exiting; sync code only
SIGTERM etc.	Signal received
uncaughtException	Thrown error not caught

COMMON MISTAKES

- Running `node` as PID 1 in Docker without an init; signals may never reach your handler (use `--init` or `tini`)
- `npm start` as the container command; npm may not forward `SIGTERM` to Node
- Async work in `process.on('exit')`; it never runs

TIPS

- Keep the deadline shorter than your orchestrator's kill timeout
- Fail health checks once shutdown starts so load balancers stop sending traffic
- Run `CMD ["node", "server.js"]` directly in Dockerfiles (Sheet 67)

SUMMARY

- `SIGTERM / SIGINT` — the signals to handle
- `server.close()` — stop accepting, drain
- `deadline` — force exit if draining hangs
- `'exit'` — final sync hook

Environment Variables & Config

CONFIGURATION FROM THE ENVIRONMENT

Apps read secrets and settings from **environment variables** so the same code runs in dev, CI, and production. Node can load a `.env` file itself with `--env-file` (or `process.loadEnvFile()` at runtime), so the `dotenv` package is optional now. Read config **once at startup**, convert types, validate, and fail fast with a clear message if something is missing.

NOTES

- Variables already set in the shell win over `.env` values
- Never commit `.env`; commit a `.env.example` with dummy values
- Node itself ignores `NODE_ENV`; frameworks and libraries read it

LOADING & VALIDATING CONFIG

```
// node --env-file=.env --env-file-if-exists=.env.local server.js
import { parseEnv } from 'node:util';

process.env.PORT;                  // '8080' (always a string)
const port = Number(process.env.PORT ?? 3000);
const debug = process.env.DEBUG === 'true';

process.loadEnvFile();             // load ./.env from code instead of a flag
process.loadEnvFile('.env.test');  // or a specific file
parseEnv('A=1\nB="two words"');    // { A: '1', B: 'two words' }

const required = ['DATABASE_URL', 'SESSION_SECRET'];
const missing = required.filter((k) => !process.env[k]);
if (missing.length) throw new Error(`Missing env: ${missing.join(', ')}`);

export const config = Object.freeze({
  port, debug, dbUrl: process.env.DATABASE_URL,
});
```

CODE EXPLAINED

- → `--env-file` — repeatable; later files override earlier ones
- → `-if-exists` — optional file, no error when missing
- → `Number()` / `=== 'true'` — explicit type conversion
- → `parseEnv` — parse .env text without touching `process.env`
- → `Object.freeze` — config can't be changed at runtime

VARIABLES NODE READS

VARIABLE	EFFECT
NODE_OPTIONS	Default CLI flags
NODE_DEBUG	Core module debug logs
NODE_EXTRA_CA_CERTS	Extra trusted CAs
NODE_NO_WARNINGS=1	Hide warnings
UV_THREADPOOL_SIZE	libuv pool size
TZ	Process time zone

.ENV SYNTAX

```
# comments are allowed
PORT=8080
DATABASE_URL="postgres://localhost/notes"
GREETING='single quotes, no expansion'
MULTI="line one
line two"
```

COMMON MISTAKES

- Reading `process.env.X` deep inside code; missing config only shows up when that path runs
- Expecting `.env` to override a variable already set in the shell; it won't
- Logging the whole `process.env`; secrets end up in log storage

TIPS

- Validate with a schema library (Zod, Valibot) for typed config
- Set `NODE_ENV=production` in production; many libraries skip dev checks
- Keep one `config.js` module that every other file imports

SUMMARY

- `--env-file` — load .env without dotenv
- `process.env` — string values only
- `validate early` — fail at startup, not mid-request
- `NODE_OPTIONS` — default flags via env

The os Module

OPERATING SYSTEM INFORMATION

The `node:os` module reports facts about the machine: CPU cores, memory, the OS name and version, network interfaces, and standard folders like the home and temp directories. Use it to size worker pools, write cross-platform paths and line endings, and include host details in logs and diagnostics.

NOTES

- `os.platform()` equals `process.platform`
- `cpus()` counts logical cores (hyperthreads included)
- `loadavg()` always returns zeros on Windows

QUERYING THE MACHINE

```
import os from 'node:os';

os.type();                    // 'Linux' | 'Darwin' | 'Windows_NT'
os.release();                 // '6.8.0-45-generic'
os.arch();                    // 'arm64'
os.cpus().length;             // 8
os.availableParallelism();    // 8 (respects CPU limits; prefer this)
os.totalmem() / 1024 ** 3;    // 16 (GB)
os.freemem();                 // free bytes
os.loadavg();                 // [0.52, 0.61, 0.70] (1, 5, 15 min)
os.uptime();                  // seconds since boot
os.homedir();                 // '/home/ada'
os.tmpdir();                  // '/tmp'
os.hostname();                // 'web-01'
os.userInfo().username;       // 'ada'
os.networkInterfaces().eth0;  // [{ address, family, mac, internal }]
os.EOL;                       // '\n' (or '\r\n' on Windows)
```

CODE EXPLAINED

- → `availableParallelism` — the right number for pool sizes
- → `totalmem` — in bytes; divide by 1024^3 for GB
- → `tmpdir` — honors `TMPDIR` / `TEMP`
- → `networkInterfaces` — find the machine's IP addresses
- → `EOL` — line ending when writing text for this OS

PLATFORM VALUES

VALUE	OS
`linux`	Linux
`darwin`	macOS
`win32`	Windows (also 64-bit)
`freebsd`	FreeBSD
`aix`	IBM AIX

ALSO USEFUL

API	USE
`os.constants.signals`	Signal numbers
`os.setPriority(pid, n)`	Change process priority
`os.devNull`	'/dev/null' or '\\.\nul'
`os.machine()`	'x86_64', 'aarch64'

COMMON MISTAKES

- Sizing a pool with `os.cpus().length` inside a container; it can report the host's cores
- Checking for `'windows'`; the value is `'win32'` even on 64-bit
- Hardcoding `/tmp` or `~`; use `os.tmpdir()` and `os.homedir()`

TIPS

- Use `os.availableParallelism() - 1` workers to leave the main thread a core
- Use `fs.mkdtemp(path.join(os.tmpdir(), 'app-'))` for temp folders
- Log `os.hostname()` to tell instances apart

SUMMARY

- `availableParallelism()` — usable cores
- `totalmem / freemem` — memory in bytes
- `homedir / tmpdir` — standard folders
- `EOL` — OS line ending

Command-Line Arguments

PARSING ARGUMENTS

`util.parseArgs()` is Node's built-in argument parser. You describe the **options** your command accepts; it returns the parsed `values` and any **positionals**, and by default throws on unknown flags. Combined with a `"bin"` entry in `package.json` and a shebang line, it's enough to ship a real command-line tool without dependencies.

NOTES

- Option types are only `'string'` and `'boolean'`; convert numbers yourself
- `strict: true` (default) rejects unknown options
- `--` ends option parsing; the rest are positionals

A SMALL CLI

```
#!/usr/bin/env node
// usage: notes add "Buy milk" --tag home -v
import { parseArgs } from 'node:util';

const { values, positionals } = parseArgs({
  allowPositionals: true,
  options: {
    tag:     { type: 'string', short: 't', multiple: true },
    verbose: { type: 'boolean', short: 'v', default: false },
    limit:   { type: 'string', default: '10' },
    help:    { type: 'boolean', short: 'h' },
  },
});
// values      → { tag: ['home'], verbose: true, limit: '10' }
// positionals → ['add', 'Buy milk']

const [command, ...rest] = positionals;
if (values.help || !command) { printHelp(); process.exit(0); }
const limit = Number(values.limit);
```

CODE EXPLAINED

- → `short` — one-letter alias: `-t home`
- → `multiple` — collects repeats into an array
- → `default` — used when the flag is absent
- → `allowPositionals` — needed for subcommands and file args
- → Missing booleans are `undefined` unless defaulted

OPTION FIELDS

FIELD	MEANING
`type`	'string' or 'boolean'
`short`	Single-letter alias
`multiple`	Allow repeats → array
`default`	Value when absent

PARSEARGS SETTINGS

SETTING	MEANING
`args`	Defaults to argv.slice(2)
`strict`	Error on unknown flags
`allowPositionals`	Accept bare args
`allowNegative`	Accept --no-color
`tokens`	Raw parse tokens

PACKAGE.JSON BIN

```
"bin": {
  "notes": "./bin/notes.js"
}
```

COMMON MISTAKES

- Forgetting the shebang; the OS tries to run the file as a shell script
- Declaring `type: 'number'`; it doesn't exist and parseArgs throws
- Testing a CLI by publishing it; use `npm link` locally first

TIPS

- Write help text by hand; parseArgs doesn't generate it
- Exit with code `2` for bad usage, `1` for runtime failures
- Reach for `commander` or `yargs` only when you need nested subcommands

SUMMARY

- `parseArgs` — built-in flag parser
- `values / positionals` — the result
- `#!/usr/bin/env node` — makes a file executable
- `"bin"` — installs your command on PATH

Reading Input

READING USER INPUT

`node:readline` reads text **line by line** from any readable stream. Its promise API, `readline/promises`, makes interactive prompts simple: `await rl.question()`. The same module processes **piped input** (`cat file | node app.js`) with `for await`. Check `process.stdin.isTTY` to tell whether a person or a pipe is on the other end.

NOTES

- Always `rl.close()`, or the open stdin keeps the process alive
- `isTTY` is `undefined` (not `false`) when input is piped
- `crlfDelay: Infinity` treats `\r\n` as one line break

PROMPTS & PIPED INPUT

```
import * as readline from 'node:readline/promises';
import { stdin as input, stdout as output } from 'node:process';
import { text } from 'node:stream/consumers';

if (input.isTTY) {                                  // a person is typing
  const rl = readline.createInterface({ input, output });
  const name = await rl.question('Name? ');         // waits for Enter
  const yes = (await rl.question('Continue? (y/N) ')).trim() === 'y';
  rl.close();
} else {                                            // cat data.txt | node app.js
  const rl = readline.createInterface({ input, crlfDelay: Infinity });
  let count = 0;
  for await (const line of rl) if (line.trim()) count++;
  console.log(count);                               // 42
}

const all = await text(process.stdin);              // whole stdin as one string
```

CODE EXPLAINED

- → `question()` — prints the prompt, resolves with the answer
- → `output` — needed for prompts; omit when only reading
- → `for await` — one iteration per line, constant memory
- → `text()` — collects a whole stream; also `json()`, `buffer()`

INTERFACE METHODS

METHOD	DOES
`question(q)`	Prompt, await answer
`close()`	Release stdin
`on('line')`	Event per line
`on('SIGINT')`	Ctrl+C while prompting
`setPrompt / prompt()`	Custom REPL prompt

NODE:STREAM/CONSUMERS

FUNCTION	RETURNS
`text(stream)`	string
`json(stream)`	Parsed JSON
`buffer(stream)`	Buffer
`arrayBuffer(stream)`	ArrayBuffer

COMMON MISTAKES

- Prompting when input is piped; `question()` reads from the pipe instead of a person
- Forgetting `rl.close()`; the program hangs after the last prompt
- Asking for passwords with `question()`; it echoes the input to the screen

TIPS

- Support both modes: read stdin when piped, prompt when interactive
- For rich prompts (select lists, hidden input) use `@inquirer/prompts`
- Handle Ctrl+C: `rl.on('SIGINT', () => process.exit(130))`

SUMMARY

- `readline/promises` — awaitable prompts
- `for await` — lines from any stream
- `isTTY` — interactive or piped
- `stream/consumers` — read a whole stream

Console & Terminal Output

WRITING TO THE TERMINAL

`console` writes to **stdout** (`log`, `info`, `table`) or **stderr** (`error`, `warn`, `trace`). Keep that split: data on stdout stays pipeable while messages go to stderr. `util.styleText` adds colors without a dependency, and `process.stdout.write` gives you output without the automatic newline, for progress lines.

NOTES

- `console.log` is synchronous to files and TTYs, async to pipes
- `styleText` respects `NO_COLOR` and skips colors when output isn't a terminal
- Exit code `0` means success to every shell and CI system

CONSOLE & STYLED OUTPUT

```
import { styleText } from 'node:util';

console.log('info');                          // stdout
console.error('oops');                         // stderr
console.table([{ id: 1, ok: true }, { id: 2, ok: false }]);   // grid
console.dir(obj, { depth: null });             // full nested object
console.log('%s has %d items', 'cart', 3);  // cart has 3 items
console.time('load'); await load(); console.timeEnd('load');  // load: 41.2ms
console.count('hit');                          // hit: 1 … hit: 2
console.group('Build'); console.log('step'); console.groupEnd();  // indented
console.assert(n > 0, 'n must be positive');   // prints only when false

console.log(styleText('green', '✔ done'));
console.log(styleText(['bold', 'red'], 'FAILED'));
process.stdout.write('Progress 40%\r');     // no newline; \r rewrites line
process.stdout.columns;                        // terminal width, e.g. 120
```

CODE EXPLAINED

- → `table` — arrays of objects become a grid
- → `depth: null` — `log` stops at 2 levels by default
- → `%s %d` — printf-style placeholders
- → `time / timeEnd` — labels must match exactly
- → `styleText` — accepts one style or an array
- → `\r` — returns the cursor to the line start

FORMAT SPECIFIERS

SPEC	FORMATS
`%s`	String
`%d / %i`	Number / integer
`%f`	Float
`%j`	JSON
`%o / %O`	Object (detailed / plain)
`%%`	Literal percent

STYLETEXT STYLES

STYLE	EXAMPLES
`Colors`	red, green, yellow, blue
`Bright`	redBright, gray
`Background`	bgRed, bgGreen
`Modifiers`	bold, dim, italic, underline

EXIT CODES

CODE	MEANING
`0`	Success
`1`	General failure
`2`	Bad usage / arguments
`130`	Interrupted (Ctrl+C)

COMMON MISTAKES

- Printing errors with `console.log`; they get mixed into piped data
- Hardcoding ANSI escape codes; they garble logs and CI output
- Using `console.log` as a production logger; it has no levels or structure (Sheet 65)

TIPS

- Print machine-readable output with a `--json` flag
- Check `process.stdout.isTTY` before drawing progress bars
- Use `console.table` for quick debugging of arrays

SUMMARY

- `log / error` — stdout vs stderr
- `table / dir` — inspect data
- `styleText` — built-in colors
- `stdout.write` — output without newline

fs API Styles

THREE WAYS TO CALL FS

Every `node:fs` operation comes in three flavors: **promise** (`fs/promises`), **callback**, and **synchronous** (`*Sync`). They do the same work; they differ in how you get the result and whether the event loop waits. Use promises by default. Sync calls are fine in CLIs, build scripts, and startup code, but never inside a server's request path.

NOTES

- All three throw or reject with the same `err.code` values
- `FileHandle` (from `open()`) is the promise API's file descriptor
- Sync calls block the whole process, including other requests

SAME READ, THREE STYLES

```
import fs from 'node:fs';
import fsp from 'node:fs/promises';

const a = await fsp.readFile('a.txt', 'utf8');      // 1. promise (default)

fs.readFile('a.txt', 'utf8', (err, b) => {          // 2. callback
  if (err) return console.error(err);
});

const c = fs.readFileSync('a.txt', 'utf8');         // 3. sync (blocks)

await using fh = await fsp.open('log.txt', 'a');    // auto-closes at scope end
await fh.appendFile('line\n');
(await fh.stat()).size;                             // 5

try { await fsp.readFile('missing.txt'); }
catch (err) {
  err.code;      // 'ENOENT'
  err.syscall;   // 'open'
  err.path;      // 'missing.txt'
}
```

CODE EXPLAINED

- → `fs/promises` — every function returns a promise
- → `callback` — error-first `(err, data)`
- → `*Sync` — returns directly, throws on error
- → `await using` — calls `fh.close()` even if code throws
- → `err.code` — branch on this, not the message

ERROR CODES

CODE	MEANING
ENOENT	No such file or folder
EEXIST	Already exists
EACCES / EPERM	Permission denied
EISDIR	Is a directory
ENOTDIR	Not a directory
ENOTEMPTY	Folder not empty
EMFILE	Too many open files

WHICH STYLE

CONTEXT	USE
Servers, APIs	Promise
Scripts, CLIs	Sync is fine
Startup config	Sync is fine
Old libraries	Callback
Big files	Streams (Sheet 26)

COMMON MISTAKES

- `readFileSync` in a request handler; every other request waits for the disk
- Opening files with `open()` and never closing them; you'll hit `EMFILE`
- Checking `err.message` text; messages change, codes don't

TIPS

- Import named functions: `import { readFile } from 'node:fs/promises'`
- Use `await using` (Node 24+) for FileHandles instead of `try/finally`
- Retry `EMFILE` / `EBUSY` with backoff, or use `graceful-fs`

SUMMARY

- `fs/promises` — default choice
- `*Sync` — scripts and startup only
- `FileHandle` — open once, many ops
- `err.code` — identify the failure

Reading Files

READING FILES

`readFile` loads a whole file into memory: give it an **encoding** to get a string, or none to get a `Buffer` of raw bytes. That's right for configs and small data. For large files, **stream** them: `createReadStream` hands you chunks, and wrapping it in `readline` gives you one line at a time with constant memory.

NOTES

- No encoding → `Buffer`; `'utf8'` → string
- Relative paths resolve from `process.cwd()`
- Paths can be strings, Buffers, or `file:` URL objects

WHOLE FILE, LINES & CHUNKS

```
import { readFile } from 'node:fs/promises';
import { createReadStream } from 'node:fs';
import { createInterface } from 'node:readline';

const text = await readFile('notes.txt', 'utf8');   // string
const img  = await readFile('logo.png');            // Buffer
const cfg  = JSON.parse(await readFile('config.json', 'utf8'));
const lines = text.split(/\r?\n/);                 // all lines in memory

const here = new URL('./data.json', import.meta.url);   // next to this file
const data = JSON.parse(await readFile(here, 'utf8'));

const rl = createInterface({                        // huge file, line by line
  input: createReadStream('big.log'), crlfDelay: Infinity,
});
for await (const line of rl) if (line.includes('ERROR')) console.log(line);

const opts = { highWaterMark: 1024 * 1024 };        // 1 MB chunks
for await (const chunk of createReadStream('big.bin', opts)) handle(chunk);
```

CODE EXPLAINED

- → `'utf8'` — decode bytes to text
- → `/\r?\n/` — split Windows and Unix line endings
- → `new URL(…, import.meta.url)` — path relative to the module
- → `readline` — memory stays flat on multi-GB files
- → `highWaterMark` — chunk size (default 64 KB)

ENCODINGS

ENCODING	USE
`utf8`	Text (default choice)
`utf16le`	Windows "Unicode" text
`latin1`	Legacy 8-bit text
`base64`	Binary as text
`hex`	Bytes as hex

CREATEREADSTREAM OPTIONS

OPTION	EFFECT
`encoding`	Emit strings
`start / end`	Byte range (inclusive)
`highWaterMark`	Chunk size
`signal`	Abort reading

WHICH READER

FILE	USE
`Small text / JSON`	readFile
`Huge, line-based`	readline + stream
`Huge binary`	createReadStream
`Part of a file`	start / end

COMMON MISTAKES

- `readFile` on a 4 GB log; it can exceed memory or the max string length
- Forgetting the encoding and printing a `<Buffer 68 65 …>`
- Paths relative to cwd when you meant relative to the file; use `import.meta.dirname`

TIPS

- Strip a UTF-8 BOM before parsing: `text.replace(/^\uFEFF/, '')`
- Use `import data from './x.json' with { type: 'json' }` for static JSON
- Pass `{ signal }` to cancel slow reads

SUMMARY

- `readFile(p, 'utf8')` — whole file as text
- `readFile(p)` — whole file as Buffer
- `readline` — line by line
- `createReadStream` — chunks, any size

Writing & Appending

WRITING FILES SAFELY

`writeFile` creates or **overwrites** a file; `appendFile` adds to the end. The `flag` option changes that behavior (for example `'wx'` fails if the file exists), and `mode` sets permissions on new files. For files other processes read, write to a temp file and `rename` it: the rename is **atomic**, so readers never see half a file.

NOTES

- Parent folders must exist; `mkdir({ recursive: true })` first
- Strings are written as UTF-8 by default
- `rename` is atomic only within the same filesystem

WRITE, APPEND & ATOMIC REPLACE

```
import { writeFile, appendFile, rename } from 'node:fs/promises';
import { createWriteStream } from 'node:fs';
import { once } from 'node:events';

await writeFile('out.txt', 'hello\n');                // create or overwrite
await writeFile('data.json', JSON.stringify(obj, null, 2));
await appendFile('app.log', `${new Date().toISOString()} start\n`);
await writeFile('new.txt', 'x', { flag: 'wx' });      // EEXIST if present
await writeFile('secret.key', key, { mode: 0o600 });  // owner-only access

await writeFile('config.json.tmp', json);             // atomic replace:
await rename('config.json.tmp', 'config.json');      // all or nothing

const out = createWriteStream('report.csv');          // many small writes
for (const r of rows) {
  if (!out.write(`${r.id},${r.total}\n`)) await once(out, 'drain');
}
out.end();
await once(out, 'finish');                            // all data flushed
```

CODE EXPLAINED

- → `null, 2` — pretty-print JSON with 2-space indent
- → `'wx'` — create-only; safe for lock files
- → `0o600` — octal permissions (read/write for owner)
- → `write()` returns `false` — buffer full, wait for `'drain'`
- → `'finish'` — fired after `end()` flushes everything

FLAGS

FLAG	MEANING
w	Write, create/truncate (default)
wx	Write, fail if exists
a	Append, create if missing
ax	Append, fail if exists
r+	Read/write, must exist

MODES

MODE	PERMISSIONS
0o644	Owner rw, others r
0o600	Owner rw only
0o755	Executable script
0o700	Private folder

COMMON MISTAKES

- Overwriting a config in place; a crash mid-write leaves it corrupted
- Calling `appendFile` thousands of times in a loop; each call opens and closes the file
- Ignoring `write()`'s return value; memory grows without limit

TIPS

- Use a stream for logs and CSV exports
- Add a trailing newline to text files so tools like `cat` and `diff` behave
- Write JSON with `JSON.stringify(data, null, 2) + '\n'`

SUMMARY

- `writeFile` — create or replace
- `appendFile` — add to the end
- `flag: 'wx'` — create only
- `tmp + rename` — atomic update

Directories & Globbing

WORKING WITH DIRECTORIES

`fs/promises` covers every directory task: **create** (with `recursive` for whole paths), **list** (with `withFileTypes` to tell files from folders, or `recursive` for the whole tree), **copy**, **move**, and **delete**. `fs.glob` matches files by pattern (like `src/**/*.test.js`), replacing the `glob` package for most uses.

NOTES

- `readdir` order is not guaranteed; sort it if order matters
- `rm` with `recursive` deletes without a recycle bin
- Glob results are relative to `cwd` unless you pass `{ cwd }`

CREATE, LIST, GLOB, DELETE

```
import { mkdir, readdir, cp, rename, rm, glob } from 'node:fs/promises';

await mkdir('out/img/thumbs', { recursive: true });   // like mkdir -p
await readdir('src');                                 // ['app.js', 'lib']

const entries = await readdir('src', { withFileTypes: true });
entries.filter((e) => e.isDirectory()).map((e) => e.name);   // ['lib']
await readdir('src', { recursive: true });   // […'lib', 'lib/db.js']

for await (const file of glob('src/**/*.test.js')) console.log(file);
const docs = await Array.fromAsync(glob('docs/*.md'));   // array of paths

await cp('template', 'site', { recursive: true });    // copy a folder
await rename('drafts', 'archive/drafts');             // move or rename
await rm('dist', { recursive: true, force: true });   // like rm -rf
```

CODE EXPLAINED

- → `recursive` on mkdir — creates parents, no error if it exists
- → `withFileTypes` — Dirent objects with `isFile()` / `isDirectory()`
- → `recursive` on readdir — paths relative to the start folder
- → `Array.fromAsync` — collects an async iterator
- → `force` — no error if the path is missing

GLOB PATTERNS

PATTERN	MATCHES
`*`	Any chars in one segment
`**`	Any number of folders
`?`	One character
`{a,b}`	Either a or b
`[0-9]`	One char from a set

DIRENT METHODS

METHOD	TRUE FOR
`isFile()`	Regular file
`isDirectory()`	Folder
`isSymbolicLink()`	Symlink
`.name / .parentPath`	Name, folder

OPERATIONS

FUNCTION	SHELL EQUIVALENT
`mkdir recursive`	mkdir -p
`readdir`	ls
`cp recursive`	cp -r
`rename`	mv
`rm recursive force`	rm -rf

COMMON MISTAKES

- `rm({ recursive: true })` on a path built from user input; one bad value deletes the wrong tree
- `mkdir` without `recursive` when parents don't exist → `ENOENT`
- Walking huge trees with `readdir` recursive into one array; use `opendir` to iterate

TIPS

- Resolve and check paths before deleting (Sheet 30)
- Use `mkdtemp(path.join(os.tmpdir(), 'job-'))` for scratch folders
- Sort listings with `names.sort((a, b) => a.localeCompare(b))`

SUMMARY

- `mkdir recursive` — create whole paths
- `readdir withFileTypes` — list with types
- `glob` — find files by pattern
- `rm recursive force` — delete a tree

File Info & Watching

METADATA, PERMISSIONS & WATCHING

`stat` returns a file's **metadata**: size, type, timestamps, and permissions. Node has no promise-based "exists" function on purpose: checking first and then acting is a race. Instead, just try the operation and handle `ENOENT`. `fs.watch` reports changes to files and folders as they happen, as an async iterator in the promise API.

NOTES

- `stat` follows symlinks; `lstat` describes the link itself
- `birthtime` isn't reliable on every filesystem
- `watch` events can arrive twice or with a missing filename

STAT, ACCESS & WATCH

```
import { stat, access, chmod, utimes, watch, constants }
  from 'node:fs/promises';

const s = await stat('photo.jpg');
s.size;              // 482113 (bytes)
s.isFile();          // true
s.isDirectory();     // false
s.mtime;             // 2026-09-30T14:02:11.000Z (last modified)
s.mode.toString(8);  // '100644'

const exists = await stat('x.txt').then(() => true, () => false);
await access('deploy.sh', constants.X_OK);      // throws if not executable
await chmod('deploy.sh', 0o755);
await utimes('a.txt', new Date(), new Date());  // like touch

const ac = new AbortController();
const opts = { recursive: true, signal: ac.signal };
for await (const { eventType, filename } of watch('src', opts)) {
  console.log(eventType, filename);             // 'change' 'app.js'
}
```

CODE EXPLAINED

- → `mtime` — use for "changed since" checks
- → `mode.toString(8)` — permission bits in octal
- → `.then(() => true, () => false)` — existence as a boolean
- → `X_OK` — check execute permission
- → `signal` — stop watching with `ac.abort()`

STATS FIELDS

FIELD	MEANING
`size`	Bytes
`mtime`	Content modified
`ctime`	Metadata changed
`atime`	Last accessed
`birthtime`	Created
`mode`	Type + permissions

ACCESS() MODES

CONSTANT	CHECKS
`F_OK`	Visible / exists
`R_OK`	Readable
`W_OK`	Writable
`X_OK`	Executable

WATCH() EVENTS

EVENT	MEANS
`change`	Content changed
`rename`	Added, removed, renamed

COMMON MISTAKES

- `if (exists) await readFile()`; the file can vanish in between, so handle `ENOENT` instead
- Reacting to every `watch` event; editors fire several per save, so debounce
- Comparing `mtime` with `===`; compare `mtimeMs` numbers

TIPS

- Use `stat(p, { throwIfNoEntry: false })` on the sync API to get `undefined`
- Use chokidar for cross-platform watching with globbing
- Use `node --watch` instead of your own watcher for dev restarts

SUMMARY

- `stat` — size, type, times
- `try + ENOENT` — the right existence check
- `chmod / access` — permissions
- `watch` — react to changes

The path Module

BUILDING PATHS CORRECTLY

`node:path` builds and takes apart file paths without string concatenation. It knows the separator for the current OS, removes `..` and duplicate slashes, and splits a path into folder, name, and extension. Use `path.join` to combine pieces and `path.resolve` to get an **absolute** path.

NOTES

- `path` only manipulates strings; it never touches the disk
- `path.posix` / `path.win32` force a specific style
- ES modules: base paths on `import.meta.dirname`

JOINING & SPLITTING PATHS

```
import path from 'node:path';
import { fileURLToPath, pathToFileURL } from 'node:url';

path.join('/app', 'src', '../lib', 'a.js');      // '/app/lib/a.js'
path.resolve('src', 'a.js');                     // '/cwd/src/a.js'
path.resolve(import.meta.dirname, 'data.json');  // next to this module
path.basename('/a/b/photo.jpg');                 // 'photo.jpg'
path.basename('/a/b/photo.jpg', '.jpg');         // 'photo'
path.extname('archive.tar.gz');                  // '.gz'
path.dirname('/a/b/photo.jpg');                  // '/a/b'
path.parse('/a/b/photo.jpg').name;               // 'photo'
path.relative('/a/b', '/a/c/d');                 // '../c/d'
path.normalize('a//b/../c/');                    // 'a/c/'
path.isAbsolute('./x');                          // false
path.sep;                                        // '/' (or '\\' on Windows)
fileURLToPath(import.meta.url);                  // '/app/src/index.js'
pathToFileURL('/tmp/a b.txt').href;              // 'file:///tmp/a%20b.txt'
```

CODE EXPLAINED

- → `join` — combines and normalizes; result can be relative
- → `resolve` — right to left until absolute, else prepends cwd
- → `basename(p, ext)` — strip a known extension
- → `parse` — `{ root, dir, base, ext, name }`
- → `fileURLToPath` — `file:` URL → OS path

JOIN VS RESOLVE

CALL	RESULT
`join('a', 'b')`	a/b
`resolve('a', 'b')`	/cwd/a/b
`join('/a', '/b')`	/a/b
`resolve('/a', '/b')`	/b

PATH.PARSE() FIELDS

FIELD	PHOTO.JPG IN /A/B
`root`	/
`dir`	/a/b
`base`	photo.jpg
`name`	photo
`ext`	.jpg

BLOCK PATH TRAVERSAL

```
const root = path.resolve('uploads');
const p = path.resolve(root, userInput);
if (!p.startsWith(root + path.sep))
  throw new Error('outside uploads');
```

COMMON MISTAKES

- Building paths with `+ '/' +`; breaks on Windows and doubles slashes
- Passing user input to `join` without checking; `../../etc/passwd` escapes your folder
- Using `new URL(import.meta.url).pathname` as a path; it keeps `%20` and breaks on Windows

TIPS

- Use `path.resolve` for anything passed to child processes or stored in config
- Use `path.posix.join` for URL paths and archive entries
- Compare paths after `path.resolve` so `./a` and `a` match

SUMMARY

- `join` — combine segments
- `resolve` — absolute path
- `basename / extname / dirname` — split a path
- `fileURLToPath` — module URL → path

Buffer Basics

RAW BYTES IN NODE

A `Buffer` is a fixed-length sequence of **bytes**, Node's type for binary data: file contents, network packets, images, hashes. It's a subclass of `Uint8Array`, so every typed-array method works too. Convert between text and bytes with an **encoding**; the byte length of a string often differs from its character length.

NOTES

- Buffers can't grow; build a list and `Buffer.concat` it
- `alloc` zero-fills; `allocUnsafe` is faster but may contain old memory
- `subarray` shares memory with the original

CREATING & CONVERTING BUFFERS

```
const a = Buffer.from('héllo', 'utf8');    // <Buffer 68 c3 a9 6c 6c 6f>
a.length;                                  // 6 (bytes, not characters)
'héllo'.length;                            // 5
a.toString();                              // 'héllo'
a.toString('base64');                      // 'aMOpbGxv'
a.toString('hex');                         // '68c3a96c6c6f'
Buffer.from('aGk=', 'base64').toString();  // 'hi'
Buffer.byteLength('héllo');                // 6, without creating a Buffer

Buffer.alloc(4);                           // <Buffer 00 00 00 00>
Buffer.from([1, 2, 255]);                  // <Buffer 01 02 ff>
Buffer.concat([a, Buffer.from('!')]);      // joined copy
const head = a.subarray(0, 2);             // bytes 0–1, same memory
a[0];                                      // 104
a.equals(Buffer.from('héllo'));            // true
a.includes('ll');                          // true
Buffer.isBuffer(a);                        // true
```

CODE EXPLAINED

- → `é` — two bytes in UTF-8, so length is 6
- → `toString(enc)` — decode bytes; default `'utf8'`
- → `byteLength` — size a string will take on disk or wire
- → `subarray` — changing `head[0]` changes `a[0]`
- → `equals` — compare contents, not identity

ENCODINGS

ENCODING	USE
`utf8`	Text (default)
`base64`	Binary in JSON / email
`base64url`	Tokens, URLs
`hex`	Hashes, debugging
`latin1`	Legacy single-byte text

CREATING

METHOD	GIVES
`from(string, enc)`	Encoded text
`from(array)`	Bytes from numbers
`from(arrayBuffer)`	View, no copy
`alloc(n)`	n zero bytes
`allocUnsafe(n)`	n uninitialized bytes

OPERATIONS

METHOD	DOES
`concat(list)`	Join buffers
`subarray(s, e)`	Slice view
`copy(target)`	Copy bytes over
`fill(v)`	Set every byte
`indexOf(x)`	Find bytes

COMMON MISTAKES

- Using string length to size binary data; use `Buffer.byteLength`
- Decoding chunks one by one with `toString()`; a multi-byte character split across chunks becomes ``
- Using `slice()` and expecting a copy; on Buffers it's a view like `subarray`

TIPS

- Decode streams with `setEncoding('utf8')` or `TextDecoderStream`, which handle split characters
- Use `crypto.timingSafeEqual` to compare secrets, not `equals`
- Log buffers as hex when debugging protocols

SUMMARY

- `Buffer` — fixed bytes, Uint8Array subclass
- `from / toString` — text ↔ bytes
- `byteLength` — bytes in a string
- `concat / subarray` — join and slice

Binary Data

READING BINARY FORMATS

Binary file formats and network protocols store numbers as fixed-size fields: 8, 16, 32, or 64 bits, signed or unsigned, in **big-endian** or **little-endian** byte order. Buffer's `read*` / `write*` methods handle each case. The standard `TypedArray`, `DataView`, `TextEncoder`, and `Blob` APIs do the same jobs in code that must also run in browsers.

NOTES

- Network protocols are usually big-endian (BE)
- x86 and ARM CPUs and most file formats are little-endian (LE)
- 64-bit integers read as `BigInt`

FIXED-WIDTH FIELDS

```
const buf = Buffer.alloc(8);
buf.writeUInt16BE(0x0102, 0);          // bytes 0-1 → 01 02
buf.writeInt32LE(-1, 2);               // bytes 2-5 → ff ff ff ff
buf.readUInt16BE(0);                   // 258
buf.readUInt16LE(0);                   // 513
buf.readUInt32LE(2);                   // 4294967295
buf.readInt32LE(2);                    // -1
buf.readBigUInt64BE(0);                // 64-bit value as BigInt

const u8 = new Uint8Array(buf.buffer, buf.byteOffset, buf.length);
const dv = new DataView(u8.buffer, u8.byteOffset, u8.byteLength);
dv.getUint16(0);                       // 258 (DataView defaults to BE)
dv.getUint16(0, true);                 // 513 (true = little-endian)

new TextEncoder().encode('hi');        // Uint8Array(2) [104, 105]
new TextDecoder().decode(new Uint8Array([104, 105]));   // 'hi'
const blob = new Blob(['hello'], { type: 'text/plain' });
await blob.text();                     // 'hello'
```

CODE EXPLAINED

- → `0x0102` — written as bytes `01 02` in BE order
- → `LE` read of `01 02` — becomes `0x0201` = 513
- → Signed vs unsigned — same bytes, different numbers
- → `buf.byteOffset` — small Buffers share a larger pool
- → `DataView` — standard API, choose endianness per call

METHOD NAMES

PART	OPTIONS
`read / write`	Direction
`UInt / Int`	Unsigned / signed
`8 / 16 / 32`	Bits
`BigUInt64 / BigInt64`	64-bit (BigInt)
`Float / Double`	32 / 64-bit IEEE
`BE / LE`	Byte order

INTEGER RANGES

TYPE	RANGE
`UInt8`	0 – 255
`Int8`	-128 – 127
`UInt16`	0 – 65,535
`Int16`	-32,768 – 32,767
`UInt32`	0 – 4,294,967,295
`Int32`	±2.1 billion

CONVERSIONS

FROM → TO	CODE
`Buffer → ArrayBuffer`	buf.buffer (+ offset)
`ArrayBuffer → Buffer`	Buffer.from(ab)
`Uint8Array → Buffer`	Buffer.from(u8.buffer)
`Blob → Buffer`	Buffer.from(await b.arrayBuffer())

COMMON MISTAKES

- Using `buf.buffer` directly; small Buffers live inside a shared 8 KB pool, so add `byteOffset`
- Writing a value outside the range; Node throws `ERR_OUT_OF_RANGE`
- Reading 64-bit IDs into a Number; values above 2^{53} lose precision

TIPS

- Write down the format's endianness at the top of your parser
- Use `buf.readUIntBE(offset, 3)` for odd sizes like 24-bit fields
- Prefer `DataView` in libraries meant for both Node and browsers

SUMMARY

- `readUInt16BE` — typed read at an offset
- `BE / LE` — byte order matters
- `DataView` — portable binary access
- `TextEncoder / Blob` — web-standard bytes

Stream Concepts

PROCESSING DATA IN PIECES

A **stream** moves data in chunks instead of all at once, so memory stays flat no matter how big the input is. There are four kinds: **Readable** (a source), **Writable** (a destination), **Duplex** (both, like a socket), and **Transform** (a Duplex that changes data, like gzip). **Backpressure** slows a fast producer down when the consumer can't keep up.

NOTES

- Files, HTTP requests/responses, sockets, and stdin/stdout are all streams
- Chunks are Buffers by default, strings with an encoding
- `objectMode: true` streams carry any JS values

READABLE → TRANSFORM → WRITABLE

```
import { createReadStream, createWriteStream } from 'node:fs';
import { createGzip } from 'node:zlib';
import { pipeline } from 'node:stream/promises';

await pipeline(                          // backpressure handled for you
  createReadStream('access.log'),        // Readable: produces chunks
  createGzip(),                          // Transform: compresses
  createWriteStream('access.log.gz'),    // Writable: consumes chunks
);

// What pipeline does for you (simplified):
const src = createReadStream('a.log');
const dst = createWriteStream('b.log');
src.on('data', (chunk) => {
  if (!dst.write(chunk)) src.pause();    // destination buffer is full
});
dst.on('drain', () => src.resume());     // destination caught up
src.on('end', () => dst.end());

src.readableHighWaterMark;               // 65536 (64 KB chunks)
```

CODE EXPLAINED

- → `pipeline` — connects streams and forwards errors
- → `write()` false — buffer passed the high-water mark
- → `pause / resume` — the backpressure mechanism
- → `'drain'` — safe to write again
- → `highWaterMark` — buffer size before backpressure kicks in

STREAM TYPES

TYPE	EXAMPLES
`Readable`	fs read, http request, stdin
`Writable`	fs write, http response, stdout
`Duplex`	TCP socket, WebSocket
`Transform`	gzip, crypto cipher, parsers

KEY EVENTS

EVENT	FIRED BY
`data`	Readable: a chunk arrived
`end`	Readable: no more data
`drain`	Writable: ready for more
`finish`	Writable: all data flushed
`close`	Both: resource released
`error`	Both: something failed

STREAM OR NOT?

DATA	USE
`Fits in memory`	readFile / text()
`Large file`	Stream
`Unknown size (upload)`	Stream
`Live / infinite`	Stream

COMMON MISTAKES

- Listening to `'data'` and writing without checking `write()`; memory grows on slow destinations
- Using `.pipe()` chains; an error in the middle leaves other streams open
- Forgetting an `'error'` handler on a bare stream; one bad file crashes the process

TIPS

- Default to `pipeline` from `node:stream/promises`
- Consume Readables with `for await` instead of `'data'` events
- Raise `highWaterMark` for fast disks or big sequential reads

SUMMARY

- `Readable / Writable` — source / destination
- `Transform` — modify data in flight
- `backpressure` — producer waits for consumer
- `pipeline` — the safe way to connect streams

Readable & Writable Streams

'data' / 'end' events · async iteration · Readable.from() · write() return value · 'drain' · end()

PRODUCING & CONSUMING STREAMS

You'll rarely subclass streams. Most of the time you **consume** Readables with `for await`, and **create** them from iterables or async generators with `Readable.from()`. For a custom destination, pass a `write` function to `new Writable()`. `Readable.fromWeb` and `toWeb` convert to and from the Web Streams that `fetch` uses.

NOTES

- Async iteration handles pause/resume for you
- Call `callback()` in `write`, or the stream stalls
- `end(chunk)` writes a final chunk and closes

CREATING READABLES & WRITABLES

```
import { Readable, Writable } from 'node:stream';

const r = Readable.from(['a', 'b', 'c']);        // from any iterable
for await (const chunk of r) console.log(chunk); // a, b, c

async function* lines() {
  for (let i = 1; i <= 3; i++) yield `row ${i}\n`;
}
Readable.from(lines()).pipe(process.stdout);     // row 1, row 2, row 3

const upper = new Writable({
  write(chunk, encoding, callback) {
    process.stdout.write(chunk.toString().toUpperCase());
    callback();                                  // ready for the next chunk
  },
});
upper.write('hello ');
upper.end('world\n');                            // HELLO WORLD
upper.on('finish', () => console.log('done'));

const res = await fetch('https://example.com/data.csv');
const nodeStream = Readable.fromWeb(res.body);   // Web → Node stream
```

CODE EXPLAINED

- → `Readable.from` — objectMode by default for non-Buffers
- → `async function*` — the easiest custom source
- → `callback(err)` — pass an error to fail the stream
- → `'finish'` — all writes processed
- → `fromWeb` — use Node stream APIs on a fetch body

READABLE API

MEMBER	DOES
`for await`	Read chunk by chunk
`read()`	Pull a chunk manually
`setEncoding(e)`	Emit strings
`pause() / resume()`	Flow control
`destroy(err)`	Abort and clean up
`readableEnded`	true after 'end'

WRITABLE API

MEMBER	DOES
`write(chunk)`	false = wait for drain
`end(chunk?)`	Finish writing
`cork() / uncork()`	Batch small writes
`writableFinished`	true after 'finish'
`destroy(err)`	Abort

READABLE HELPERS

METHOD	RETURNS
`map(fn)`	New Readable
`filter(fn)`	New Readable
`take(n)`	First n chunks
`toArray()`	Promise of all chunks

COMMON MISTAKES

- Never calling `callback` in a custom `write`; the stream stops after one chunk
- Mixing `for await` with `'data'` listeners on the same stream
- Breaking out of `for await` and expecting the source to stay open; it's destroyed

TIPS

- Generate large CSV or JSON exports with an async generator + `Readable.from`
- Use `stream/consumers` (`text`, `json`) to read a whole stream
- Add `concurrency` to `map()` to process chunks in parallel

SUMMARY

- `for await` — consume a Readable
- `Readable.from` — make one from data
- `new Writable({ write })` — custom destination
- `fromWeb / toWeb` — bridge to fetch streams

pipeline() & Transforms

CONNECTING STREAMS SAFELY

`pipeline()` from `node:stream/promises` connects any number of streams, propagates **backpressure**, and if any stage fails, **destroys every stream** and rejects. Middle stages can be `Transform` streams, `zlib` compressors, or plain **async generator functions**, which are the simplest way to write a transform.

NOTES

- `.pipe()` doesn't forward errors; `pipeline` does
- A generator stage receives the previous stage as `source`
- Pass `{ signal }` as the last argument to make it cancellable

PIPELINE, TRANSFORMS & COMPRESSION

```
import { pipeline } from 'node:stream/promises';
import { Transform } from 'node:stream';
import { createReadStream, createWriteStream } from 'node:fs';
import { createGzip, createGunzip } from 'node:zlib';

const upper = new Transform({
  transform(chunk, enc, cb) { cb(null, chunk.toString().toUpperCase()); },
});
await pipeline(createReadStream('in.txt'), upper, createGzip(),
  createWriteStream('out.txt.gz'));           // any error rejects

await pipeline(
  createReadStream('data.csv.gz'),
  createGunzip(),
  async function* (source) {                  // generator as a transform
    for await (const chunk of source)
      yield chunk.toString().replaceAll(';', ',');
  },
  createWriteStream('data.csv'),
  { signal: AbortSignal.timeout(60_000) },    // cancel after 60 s
);
```

CODE EXPLAINED

- → `cb(null, data)` — push transformed data downstream
- → `createGzip` — a Transform stream from zlib
- → `source` — async iterable of the previous stage's chunks
- → `yield` — sends a chunk to the next stage
- → `signal` — aborting destroys all stages

ZLIB STREAMS

FUNCTION	FORMAT
`createGzip / createGunzip`	.gz
`createBrotliCompress`	.br (best for text)
`createBrotliDecompress`	.br
`createDeflate / createInflate`	zlib / deflate
`createZstdCompress`	.zst (newer)

ONE-SHOT ZLIB

CALL	USE
`gzipSync(buf)`	Small data, sync
`promisify(gzip)`	Small data, async
`gunzipSync(buf)`	Decompress
`brotliCompressSync`	Brotli, small data

STREAM/PROMISES

FUNCTION	DOES
`pipeline(...)`	Connect + await all
`finished(stream)`	Await end or error

COMMON MISTAKES

- `a.pipe(b).pipe(c)` in production; an error in `b` leaves `a` and `c` open
- Using `replaceAll` on chunks for multi-character patterns; a match can be split between chunks
- Gzipping already-compressed files (JPEG, MP4); it wastes CPU for no gain

TIPS

- Serve precompressed `.br` / `.gz` files instead of compressing per request
- Split text into lines first (readline) before line-based transforms
- Use `objectMode` transforms to parse records, not raw bytes

SUMMARY

- `pipeline` — connect, propagate errors
- `Transform` — change chunks in flight
- `async generator` — easiest transform
- `zlib` — gzip / brotli streams

Web Streams

THE WEB-STANDARD STREAMS

Node also implements the **WHATWG streams** used by browsers, `fetch`, and edge runtimes: `ReadableStream`, `WritableStream`, and `TransformStream`. They have a different API from Node streams but the same ideas. Use them for code that must run everywhere, and convert with `Readable.toWeb()` / `fromWeb()` when mixing with Node APIs.

NOTES

- `fetch` request and response bodies are Web streams
- Web streams are globals; no import needed
- `pipeTo` / `pipeThrough` replace `pipeline`

READABLESTREAM, TRANSFORMSTREAM & BRIDGES

```
import fs from 'node:fs';
import { Readable, Writable } from 'node:stream';

const rs = new ReadableStream({
  start(controller) {
    controller.enqueue('a'); controller.enqueue('b'); controller.close();
  },
});
for await (const chunk of rs) console.log(chunk);   // a, b

const upper = new TransformStream({
  transform(chunk, controller) { controller.enqueue(chunk.toUpperCase()); },
});

const res = await fetch('https://example.com/readme.txt');
await res.body
  .pipeThrough(new TextDecoderStream())             // bytes → strings
  .pipeThrough(upper)
  .pipeTo(Writable.toWeb(process.stdout));          // Node → Web writable

const web = Readable.toWeb(fs.createReadStream('a.txt'));   // Node → Web
const gz = web.pipeThrough(new CompressionStream('gzip'));
```

CODE EXPLAINED

- → `controller.enqueue` — push a chunk
- → `controller.close()` — end of data
- → `pipeThrough` — attach a TransformStream, returns the readable side
- → `pipeTo` — sends to a WritableStream, returns a promise
- → `CompressionStream` — built-in gzip / deflate

NODE VS WEB TERMS

NODE	WEB
`Readable`	ReadableStream
`Writable`	WritableStream
`Transform`	TransformStream
`pipeline()`	pipeThrough / pipeTo
`push()`	controller.enqueue()
`destroy()`	cancel() / abort()

BUILT-IN TRANSFORMS

CLASS	DOES
`TextDecoderStream`	Bytes → text
`TextEncoderStream`	Text → bytes
`CompressionStream`	gzip / deflate
`DecompressionStream`	Inflate

BRIDGES

CALL	CONVERTS
`Readable.toWeb(r)`	Node → Web
`Readable.fromWeb(rs)`	Web → Node
`Writable.toWeb(w)`	Node → Web
`Duplex.toWeb(d)`	→ { readable, writable }

COMMON MISTAKES

- Reading `res.body` twice; a stream can only be consumed once (use `res.clone()`)
- Passing a Node stream where a Web stream is expected, e.g. to `new Response()`
- Forgetting `await` on `pipeTo`; errors become unhandled rejections

TIPS

- Write portable libraries against Web streams; convert at the Node edge
- Use `new Response(stream).text()` to collect a Web stream
- Stick with Node streams for fs and sockets; they're faster there

SUMMARY

- `ReadableStream` — web-standard source
- `pipeThrough / pipeTo` — connect stages
- `TextDecoderStream` — decode safely
- `toWeb / fromWeb` — convert between worlds

HTTP Server Basics

THE HTTP MODULE

`node:http` is the server every Node framework is built on. `createServer` takes one function that runs for **every request**, receiving `req` (an IncomingMessage, a Readable) and `res` (a ServerResponse, a Writable). You set the status and headers, write the body, and **must** call `res.end()`. Routing, parsing, and middleware are up to you, which is why frameworks exist.

NOTES

- Header names in `req.headers` are lowercase
- Headers must be set before the first body write
- Default `requestTimeout` is 300 s; `headersTimeout` 60 s

A MINIMAL SERVER

```
import { createServer } from 'node:http';

const server = createServer((req, res) => {
  if (req.method === 'GET' && req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
    return res.end('<h1>Home</h1>');
  }
  if (req.url === '/api/health') {
    res.setHeader('Content-Type', 'application/json');
    return res.end(JSON.stringify({ ok: true }));   // 200 by default
  }
  res.statusCode = 404;
  res.end('Not found');
});

server.listen(3000, () => console.log('http://localhost:3000'));
server.keepAliveTimeout = 65_000;   // above the load balancer's idle timeout
server.on('error', (err) => {
  if (err.code === 'EADDRINUSE') console.error('Port 3000 is taken');
});
```

CODE EXPLAINED

- → `req.url` — path + query string, e.g. `/a?b=1`
- → `writeHead` — status and headers in one call
- → `return res.end()` — prevents writing twice
- → `statusCode` — set before `end()`
- → `keepAliveTimeout` — avoids 502s behind AWS ALB / nginx

REQUEST (REQ)

MEMBER	VALUE
`method`	'GET', 'POST' …
`url`	'/path?query'
`headers`	Lowercase object
`httpVersion`	'1.1'
`socket.remoteAddress`	Client IP
`on('data' / 'end')`	Body chunks

RESPONSE (RES)

MEMBER	DOES
`statusCode`	Set status
`setHeader(k, v)`	Set a header
`writeHead(s, h)`	Status + headers
`write(chunk)`	Body chunk
`end(data?)`	Finish response
`headersSent`	true once sent

SERVER TIMEOUTS

PROPERTY	DEFAULT
`requestTimeout`	300 000 ms
`headersTimeout`	60 000 ms
`keepAliveTimeout`	5 000 ms
`maxHeadersCount`	2 000

COMMON MISTAKES

- Forgetting `res.end()`; the client waits until it times out
- Writing after `end()` → `ERR_STREAM_WRITE_AFTER_END`
- Comparing `req.url === '/search'` when a query string is attached; parse it (Sheet 38)

TIPS

- Bind to `0.0.0.0` in containers; `localhost` isn't reachable from outside
- Read the port from `process.env.PORT`
- Use a framework (Express, Fastify) for anything beyond a few routes

SUMMARY

- `createServer(fn)` — one handler per request
- `req` — method, url, headers, body stream
- `res.writeHead / end` — send the reply
- `listen(port)` — start accepting

Requests, Bodies & URLs

PARSING URLS & BODIES

`req.url` is just a string like `/search?q=node`. Parse it with the WHATWG URL class, which needs a base, and read query values from `url.searchParams`. The request **body** arrives as a stream; `node:stream/consumers` collects it into text, JSON, or a Buffer. Always cap the body size before reading it.

NOTES

- `searchParams` values are strings, already decoded
- Query keys can repeat: use `getAll`
- Check `Content-Type` before parsing JSON

QUERY STRINGS & JSON BODIES

```
import { createServer } from 'node:http';
import { json } from 'node:stream/consumers';

createServer(async (req, res) => {
  const url = new URL(req.url, `http://${req.headers.host}`);
  url.pathname;                          // '/search'
  url.searchParams.get('q');             // 'node streams'
  url.searchParams.getAll('tag');        // ['a', 'b']
  Number(url.searchParams.get('page') ?? 1);   // 2

  if (req.method === 'POST') {
    if (Number(req.headers['content-length']) > 1_000_000) {
      res.writeHead(413); return res.end('Too large');
    }
    try {
      const body = await json(req);    // read + parse the whole body
      res.writeHead(201, { 'Content-Type': 'application/json' });
      return res.end(JSON.stringify({ received: body }));
    } catch { res.writeHead(400); return res.end('Bad JSON'); }
  }
  res.end();
}).listen(3000);
```

CODE EXPLAINED

→ Base URL — required because `req.url` is relative

→ `get` — first value or `null`

→ `413` — Payload Too Large

→ `json(req)` — throws on invalid JSON, hence `try`

→ `content-length` can be absent (chunked); a stream limit is safer

URL PARTS

PROPERTY	HTTPS://A.DEV:8080/P?Q=1#TOP
protocol	'https:'
host	'a.dev:8080'
hostname	'a.dev'
pathname	'/p'
search	'?q=1'
hash	'#top'

URLSEARCHPARAMS

METHOD	DOES
get(k)	First value / null
getAll(k)	All values
has(k)	Key present?
set / append	Replace / add
toString()	'q=1&tag=a'

COMMON STATUSES

CODE	MEANING
201	Created
204	No content
400	Bad request
404	Not found
413	Body too large
422	Validation failed

COMMON MISTAKES

- Splitting `req.url` on `?` and `&` by hand; misses decoding and repeated keys
- Reading the full body without a size cap; one client can exhaust memory
- Trusting `Host` to build absolute URLs; it's client-controlled

TIPS

- Use `URL.canParse(str)` to validate URLs without try/catch
- Build query strings with `new URLSearchParams(obj).toString()`
- Validate parsed bodies with a schema before using them

SUMMARY

- `new URL(req.url, base)` — parse a request URL
- `searchParams` — query values
- `stream/consumers` — read the body
- `size limit` — always cap uploads

fetch & Undici

MAKING HTTP REQUESTS

`fetch` is built into Node and works like the browser version. It's powered by **Undici**, Node's HTTP client, which you can import directly for connection pools, proxies, and lower-level control. `fetch` only rejects on **network** errors; a 404 or 500 still resolves, so check `res.ok` yourself.

NOTES

- Undici 8 ships with Node 26
- No timeout by default; pass a `signal`
- The body can be read once: `json()`, `text()`, `arrayBuffer()`, or `blob()`

GET, POST & UPLOADS

```
const res = await fetch('https://api.example.com/notes?limit=10', {
  headers: { Authorization: `Bearer ${token}` },
  signal: AbortSignal.timeout(5000),            // give up after 5 s
});
res.ok;                                         // true for 200-299
res.status;                                     // 200
res.headers.get('content-type');                // 'application/json'
const notes = await res.json();

const created = await fetch('https://api.example.com/notes', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ title: 'Buy milk' }),
});
if (!created.ok) throw new Error(`HTTP ${created.status}`);

const form = new FormData();
form.append('file', new Blob([bytes], { type: 'image/jpeg' }), 'photo.jpg');
await fetch(uploadUrl, { method: 'POST', body: form });   // multipart
```

CODE EXPLAINED

- → `AbortSignal.timeout` — rejects with `TimeoutError`
- → `res.ok` — the check fetch won't do for you
- → `JSON.stringify` + header — fetch doesn't encode objects
- → `FormData` — sets the multipart boundary itself
- → Don't set `Content-Type` for FormData manually

FETCH OPTIONS

OPTION	VALUE
`method`	'GET', 'POST' …
`headers`	Object or Headers
`body`	string, FormData, Blob, stream
`signal`	AbortSignal
`redirect`	'follow' / 'manual' / 'error'
`keepalive`	Reuse connection hint

UNDICI EXTRAS

API	USE
`Agent`	Pool size, timeouts
`ProxyAgent`	Route through a proxy
`setGlobalDispatcher`	Apply to every fetch
`request()`	Faster low-level client
`NODE_USE_ENV_PROXY=1`	Honor HTTP(S)_PROXY

COMMON MISTAKES

- Assuming `fetch` throws on 4xx/5xx; it resolves, so check `res.ok`
- Calling `res.json()` then `res.text()`; the body is already used
- No timeout on outbound calls; one slow API stalls your requests

TIPS

- Wrap fetch in a small helper that adds timeout, `ok` check, and JSON parsing
- Stream big downloads: `Readable.fromWeb(res.body)` into a file
- Retry only idempotent requests (GET, PUT), with backoff

SUMMARY

- `fetch` — built-in HTTP client
- `res.ok` — check status yourself
- `AbortSignal.timeout` — set a deadline
- `undici` — pools, proxies, speed

HTTPS & TLS

ENCRYPTED SERVERS & CLIENTS

`node:https` is `node:http` plus TLS: pass a private **key** and **certificate** and everything else works the same. `node:http2` adds HTTP/2 (multiplexed streams over one connection). In production, TLS is often handled by a reverse proxy or load balancer instead, with Node serving plain HTTP behind it.

NOTES

- `cert` should contain the full chain (leaf + intermediates)
- Browsers require TLS for HTTP/2
- Clients reject untrusted certs by default

HTTPS & HTTP/2 SERVERS

```
import https from 'node:https';
import http2 from 'node:http2';
import { readFileSync } from 'node:fs';

const tls = {
  key:  readFileSync('certs/key.pem'),
  cert: readFileSync('certs/fullchain.pem'),   // cert + intermediates
  minVersion: 'TLSv1.2',
};
https.createServer(tls, (req, res) => res.end('secure\n')).listen(8443);

const h2 = http2.createSecureServer({ ...tls, allowHTTP1: true });
h2.on('request', (req, res) => res.end(`HTTP/${req.httpVersion}`));
h2.listen(8444);

const agent = new https.Agent({ ca: readFileSync('company-ca.pem') });
https.get('https://intranet.local', { agent }, (res) => {
  console.log(res.statusCode);                 // 200
});
```

CODE EXPLAINED

- → `key / cert` — PEM files as Buffers or strings
- → `minVersion` — refuse old TLS versions
- → `allowHTTP1` — same server answers HTTP/1.1 clients
- → `Agent({ ca })` — trust a private CA for these requests

TRUST SETTINGS

SETTING	EFFECT
`NODE_EXTRA_CA_CERTS=file`	Add CAs to bundled list
`--use-system-ca`	Trust the OS cert store
`ca: [...]`	Per-agent CAs
`rejectUnauthorized: false`	Disable checks (never in prod)

TLS OPTIONS

OPTION	USE
`key / cert`	Server identity
`ca`	Trusted CAs
`minVersion`	e.g. 'TLSv1.2'
`requestCert`	Ask for client cert (mTLS)
`SNICallback`	Cert per hostname

SELF-SIGNED DEV CERT

```
openssl req -x509 -newkey rsa:2048 \
  -nodes -days 365 \
  -keyout key.pem -out cert.pem \
  -subj "/CN=localhost"
```

COMMON MISTAKES

- `NODE_TLS_REJECT_UNAUTHORIZED=0` to "fix" a cert error; it disables TLS checks for every request
- Serving only the leaf cert; some clients fail with "unable to verify the first certificate"
- Reading cert files once and never reloading them after renewal

TIPS

- Use `mkcert` to create locally trusted dev certificates
- Terminate TLS at a proxy (nginx, Caddy, cloud LB) unless you need it in Node
- Use `--use-system-ca` on corporate networks with inspection proxies

SUMMARY

- `https.createServer(tls)` — TLS server
- `http2.createSecureServer` — HTTP/2
- `Agent({ ca })` — trust private CAs
- `fullchain.pem` — serve the whole chain

WebSockets

REAL-TIME CONNECTIONS

A **WebSocket** is a long-lived, two-way connection over one TCP socket, upgraded from an HTTP request. Node has a built-in WebSocket **client** with the browser API. For a **server**, use the `ws` package (or a framework plugin built on it). Plan for dropped connections: send periodic pings and terminate sockets that stop answering.

NOTES

- `wss://` is WebSocket over TLS
- Messages are strings or binary; JSON is common
- Load balancers must allow the HTTP `Upgrade`

CLIENT (BUILT IN) & SERVER (WS)

```
import { WebSocketServer } from 'ws';                // npm i ws

const ws = new WebSocket('wss://echo.example.com');  // client, global
ws.addEventListener('open', () => ws.send(JSON.stringify({ type: 'hi' })));
ws.addEventListener('message', (e) => console.log(JSON.parse(e.data)));
ws.addEventListener('close', (e) => console.log('closed', e.code));

const wss = new WebSocketServer({ port: 8080 });     // server
wss.on('connection', (socket, req) => {
  socket.isAlive = true;
  socket.on('pong', () => { socket.isAlive = true; });
  socket.on('message', (data) => {                    // broadcast
    for (const c of wss.clients)
      if (c.readyState === WebSocket.OPEN) c.send(data);
  });
});

setInterval(() => {                                   // heartbeat
  for (const s of wss.clients) {
    if (!s.isAlive) { s.terminate(); continue; }
    s.isAlive = false; s.ping();
  }
}, 30_000);
```

CODE EXPLAINED

- → `addEventListener` — same API as browsers
- → `req` — the upgrade request: read cookies, auth here
- → `wss.clients` — Set of open sockets
- → `ping / pong` — detects dead peers
- → `terminate` — hard close, no handshake

READYSTATE

VALUE	MEANING
`0 CONNECTING`	Handshake in progress
`1 OPEN`	Can send
`2 CLOSING`	Closing handshake
`3 CLOSED`	Done

CLOSE CODES

CODE	MEANING
`1000`	Normal close
`1001`	Going away (shutdown)
`1006`	Dropped, no close frame
`1008`	Policy violation
`1011`	Server error

SHARE AN HTTP PORT

OPTION	USE
`{ server }`	Attach to http.Server
`{ noServer: true }`	Handle 'upgrade' yourself
`{ path: '/ws' }`	Only this path

COMMON MISTAKES

- Skipping heartbeats; half-open connections pile up until memory runs out
- Authenticating after connect; check the token during the upgrade request
- Broadcasting without checking `readyState`; sends to closing sockets throw

TIPS

- Reconnect clients with exponential backoff
- Use Redis pub/sub or similar to broadcast across multiple server instances
- Consider Server-Sent Events for one-way server → client updates

SUMMARY

- `WebSocket` — built-in client
- `ws` — the standard server package
- `ping / pong` — keep connections honest
- `wss://` — always in production

TCP, UDP & DNS

LOWER-LEVEL NETWORKING

`node:net` provides raw **TCP** servers and sockets: a byte stream with no HTTP on top. `node:dgram` provides **UDP**: individual datagrams, no connection, no delivery guarantee. `node:dns` resolves names; `lookup` uses the OS resolver (and `/etc/hosts`), while the `resolve*` functions query DNS servers directly.

NOTES

- A TCP socket is a Duplex stream
- TCP has no message boundaries; you define framing
- UDP messages can be lost, duplicated, or reordered

TCP, UDP & DNS

```
import net from 'node:net';
import dgram from 'node:dgram';
import dns from 'node:dns/promises';

net.createServer((socket) => {                  // TCP echo server
  socket.write('hi\n');
  socket.on('data', (buf) => socket.write(buf));
  socket.on('end', () => console.log('client left'));
}).listen(7000);

const client = net.connect({ host: 'localhost', port: 7000 });
client.on('data', (d) => console.log(d.toString()));   // 'hi'

const udp = dgram.createSocket('udp4');
udp.on('message', (msg, rinfo) => console.log(msg.toString(), rinfo.port));
udp.bind(9000, () => udp.send('ping', 9000, '127.0.0.1'));

await dns.lookup('nodejs.org');      // { address: '104.20.…', family: 4 }
await dns.resolve4('nodejs.org');             // ['104.20.…', …]
await dns.resolveMx('gmail.com');             // [{ exchange, priority }, …]
```

CODE EXPLAINED

- → `createServer(fn)` — runs per client connection
- → `'end'` — the other side finished sending
- → `rinfo` — sender's address and port
- → `bind` callback — send once the socket is ready
- → `lookup` — same answer the OS gives other programs

TCP VS UDP

FEATURE	TCP / UDP
Connection	Yes / No
Ordered	Yes / No
Reliable	Yes / No
Boundaries	Stream / Messages
Uses	HTTP, DB / DNS, games, metrics

DNS FUNCTIONS

FUNCTION	RETURNS
`lookup(host)`	OS answer, one address
`resolve4 / resolve6`	A / AAAA records
`resolveMx`	Mail servers
`resolveTxt`	TXT records
`reverse(ip)`	Hostnames for an IP

SOCKET EVENTS

EVENT	WHEN
`connect`	Client connected
`data`	Bytes received
`end`	Peer finished writing
`close`	Fully closed
`timeout`	Idle (setTimeout)

COMMON MISTAKES

- Treating each `'data'` event as one message; TCP can split or merge writes
- No `'error'` handler on sockets; a reset connection (`ECONNRESET`) crashes the app
- Using `resolve4` and expecting `/etc/hosts` entries; only `lookup` reads them

TIPS

- Frame TCP messages with a length prefix or newline delimiter
- Use `socket.setKeepAlive(true)` for long-lived connections
- Use `net.isIP(str)` to validate IP addresses

SUMMARY

- `net` — raw TCP servers and clients
- `dgram` — UDP datagrams
- `dns.lookup` — OS name resolution
- `framing` — your job on TCP

Error Patterns in Node

ERRORS THE NODE WAY

Node errors carry more than a message. **System errors** from fs, net, and dns have a `code` like `ENOENT` plus `syscall` and `path`; Node's own errors use `ERR_*` codes. Give your own errors a `name` and `code`, and when you rethrow, wrap the original as `cause` so the root problem survives. Separate **operational** errors (expected; handle them) from **programmer** errors (bugs; crash and fix).

NOTES

- Branch on `err.code`, never on `err.message`
- `cause` shows up in stack traces and logs
- `AggregateError` holds several errors in `.errors`

CUSTOM ERRORS & WRAPPING

```
import { readFile } from 'node:fs/promises';

class NotFoundError extends Error {
  constructor(resource, id, options) {
    super(`${resource} ${id} not found`, options);   // options.cause
    this.name = 'NotFoundError';
    this.code = 'E_NOT_FOUND';
    this.status = 404;
  }
}

try {
  await readFile('config.json', 'utf8');
} catch (err) {
  if (err.code !== 'ENOENT') {                       // ENOENT: use defaults
    throw new Error('Could not load config', { cause: err });
  }
}

new NotFoundError('User', 7).message;   // 'User 7 not found'
throw new AggregateError([e1, e2], '2 jobs failed');   // e.errors → [e1, e2]
```

CODE EXPLAINED

- → `super(msg, options)` — passes `cause` through
- → `name` — shown at the top of the stack trace
- → `code` — stable identifier for callers to check
- → `{ cause: err }` — wrap without losing the original

OPERATIONAL VS PROGRAMMER

KIND	EXAMPLES
Operational	File missing, timeout, bad input
Programmer	undefined is not a function
Operational →	Handle, retry, return 4xx
Programmer →	Log, crash, fix the bug

SYSTEM ERROR FIELDS

FIELD	EXAMPLE
code	'ECONNREFUSED'
syscall	'connect'
path	'/data/x.json'
address / port	'127.0.0.1' / 5432

COMMON ERR_ CODES

CODE	CAUSE
ERR_INVALID_ARG_TYPE	Wrong argument type
ERR_HTTP_HEADERS_SENT	Header after body
ERR_MODULE_NOT_FOUND	Bad import path
ERR_STREAM_PREMATURE_CLOSE	Stream ended early

COMMON MISTAKES

- Throwing strings (`throw 'failed'`); no stack trace and no `code`
- Wrapping without `cause`; the real reason disappears from logs
- Catching everything and continuing; programmer errors leave the process in an unknown state

TIPS

- Keep a small set of error classes (NotFound, Validation, Conflict) and map them to HTTP statuses in one place
- Log errors as objects, not `err.message`, so `stack` and `cause` are kept
- Use `err.code` constants from the docs in tests

SUMMARY

- `err.code` — what to branch on
- `class X extends Error` — custom errors
- `{ cause }` — wrap and keep context
- `operational vs programmer` — handle vs crash

Uncaught Errors & Rejections

WHEN ERRORS ESCAPE

An error nobody catches emits `'uncaughtException'`; a rejected promise nobody handles emits `'unhandledRejection'`. Both crash the process by default, which is correct: after an unknown error, the app's state can't be trusted. Use these handlers only to **log and exit**, and let a supervisor (Docker, systemd, Kubernetes, PM2) start a fresh process.

NOTES

- Without a handler: stack trace to stderr, exit code `1`
- `origin` tells you whether it came from a rejection
- Errors thrown inside callbacks can't be caught by an outer `try`

LAST-RESORT HANDLERS

```
process.on('uncaughtException', (err, origin) => {
  logger.fatal({ err, origin }, 'uncaught exception');
  process.exitCode = 1;
  setTimeout(() => process.exit(1), 1000).unref();   // let logs flush
});

process.on('unhandledRejection', (reason) => {
  throw reason;                       // turn it into an uncaughtException
});

// Where escaped errors come from:
setTimeout(() => { throw new Error('boom'); });     // no caller to catch it
emitter.emit('error', new Error('x'));              // no 'error' listener
async function save() { throw new Error('db down'); }
save();                                             // no await, no .catch()
stream.pipe(dest);                            // pipe errors aren't forwarded

try {
  setTimeout(() => { throw new Error('nope'); });   // ✖ try can't catch this
} catch {}
```

CODE EXPLAINED

- → `logger.fatal` — log with full stack before dying
- → `exit` after a delay — gives async log transports time
- → `throw reason` — one code path for both kinds
- → `save()` without `await` — a floating promise
- → `try` only catches errors on the current call stack

SOURCES & FIXES

SOURCE	FIX
`Thrown in a callback`	try/catch inside it
`Floating promise`	await or .catch()
`Emitter 'error'`	Add a listener
`.pipe() chains`	stream pipeline()
`JSON.parse on input`	Wrap in try/catch

RELATED FLAGS

FLAG	EFFECT
`--unhandled-rejections=strict`	Crash immediately
`--trace-uncaught`	Show where it was thrown
`--abort-on-uncaught-exception`	Core dump

RESTART WITH

TOOL	SETTING
`Docker`	--restart unless-stopped
`systemd`	Restart=on-failure
`Kubernetes`	Pod restartPolicy
`PM2`	Restarts by default

COMMON MISTAKES

- Logging in `uncaughtException` and continuing to serve requests; resources may be half-open or corrupted
- `process.on('unhandledRejection', () => {})` to hide warnings; real failures go silent
- Relying on the handler instead of fixing the floating promise it reported

TIPS

- Turn on `no-floating-promises` in typescript-eslint
- Send fatal errors to your error tracker (Sentry, etc.) before exit
- Test crash behavior: make sure your container actually restarts

SUMMARY

- `uncaughtException` — last chance to log
- `unhandledRejection` — floating promise failed
- `log + exit` — the only safe response
- `supervisor` — restarts the process

Debugging

STEPPING THROUGH CODE

`--inspect` starts the **V8 inspector**, the same debugging protocol Chrome DevTools uses. Attach from Chrome (`chrome://inspect`), VS Code, or any JetBrains IDE to set breakpoints, step through code, inspect variables, and profile. A `debugger;` statement pauses execution when a debugger is attached and does nothing otherwise.

NOTES

- Default address is `127.0.0.1:9229`
- Anyone who can reach the port can run code in your process
- VS Code's JavaScript Debug Terminal auto-attaches to any `node` you run

STARTING THE DEBUGGER

```
node --inspect app.js              # run with debugger on 127.0.0.1:9229
node --inspect-brk app.js          # pause before the first line
node --inspect-wait app.js         # wait for a debugger, then run
node --inspect=9230 app.js         # different port (two processes)
node --watch --inspect app.js      # restart on change, reattach
node --inspect-brk --test          # debug tests
node inspect app.js                # built-in terminal debugger
kill -USR1 48213                   # enable the inspector in a running process

# then open chrome://inspect → "Open dedicated DevTools for Node"
```

CODE EXPLAINED

- → `--inspect-brk` — set breakpoints before any code runs
- → `--inspect-wait` — like brk, but doesn't pause on line 1
- → `=9230` — needed when debugging two processes
- → `node inspect` — no GUI required (SSH sessions)
- → `SIGUSR1` — attach to a process started without flags

TERMINAL DEBUGGER

COMMAND	DOES
`cont / c`	Continue
`next / n`	Step over
`step / s`	Step into
`out / o`	Step out
`repl`	Evaluate in scope
`watch('x')`	Watch an expression
`setBreakpoint(5)`	Break at line 5

DEVTOOLS PANELS

PANEL	USE
`Sources`	Breakpoints, stepping
`Console`	Evaluate in paused scope
`Memory`	Heap snapshots
`Performance`	CPU profile

.VSCODE/LAUNCH.JSON

```
{ "type": "node", "request": "launch",
  "name": "Debug app",
  "program": "${workspaceFolder}/app.js"
  "envFile": "${workspaceFolder}/.env" }
```

COMMON MISTAKES

- › `--inspect=0.0.0.0` on a server; anyone on the network gets remote code execution
- › Forgetting `--enable-source-maps` when debugging compiled code
- › Leaving `debugger;` statements in committed code

TIPS

- › Use conditional breakpoints instead of adding `if` + `debugger`
- › Use logpoints to print without editing code
- › Tunnel to a remote debugger over SSH: `ssh -L 9229:localhost:9229 host`

SUMMARY

- › `--inspect` — enable the debugger
- › `--inspect-brk` — pause at start
- › `chrome://inspect` — attach from Chrome
- › `debugger;` — code breakpoint

Diagnostics & Tracing

SEEING INSIDE A RUNNING PROCESS

Beyond a debugger, Node has built-in tools for diagnosing problems in place. `util.inspect` prints any value in full; `util.debuglog` adds debug logging that's off unless `NODE_DEBUG` enables it; process **warnings** report deprecations and leaks; `diagnostics_channel` lets tracing tools observe libraries; and **diagnostic reports** capture a JSON snapshot of the process.

NOTES

- `NODE_DEBUG=http,net` also enables core module logs
- Warnings print to stderr once per process
- APM agents (OpenTelemetry, Datadog) subscribe to diagnostics channels

INSPECT, DEBUG LOGS & CHANNELS

```
import util from 'node:util';
import dc from 'node:diagnostics_channel';

util.inspect(deep, { depth: null, colors: true, breakLength: 100 });
const debug = util.debuglog('payments');   // NODE_DEBUG=payments to enable
debug('charged %d cents', 1250);           // PAYMENTS 4821: charged 1250 cents

process.on('warning', (w) => log.warn({ name: w.name, msg: w.message }));
process.emitWarning('Option "x" is old', 'DeprecationWarning', 'DEP_APP_001');

const ch = dc.channel('app:db.query');
ch.subscribe(({ sql, ms }) => metrics.observe('db_ms', ms));
if (ch.hasSubscribers) ch.publish({ sql, ms: 12 });   // near-zero cost if not

process.report.writeReport();              // report.20261002.123456.json
```

CODE EXPLAINED

- → `depth: null` — print every nesting level
- → `debuglog` — section name is uppercased in the output
- → `'warning'` event — route warnings into your logger
- → `hasSubscribers` — skip building data no one reads
- → `writeReport` — stack, heap stats, handles, env

DIAGNOSTIC FLAGS

FLAG	SHOWS
`--trace-warnings`	Stack for each warning
`--trace-deprecation`	Stack for deprecations
`--throw-deprecation`	Make them errors
`--stack-trace-limit=50`	Longer stacks
`--report-on-fatalerror`	Report on crash
`--report-on-signal`	Report on SIGUSR2

NODE_DEBUG SECTIONS

VALUE	LOGS
`http`	Requests, sockets
`net`	TCP connections
`fs`	File operations
`module`	Module resolution
`tls`	Handshakes

COMMON MISTAKES

- Logging objects with `console.log` and seeing `[Object]`; use `util.inspect(x, { depth: null })`
- Silencing all warnings with `--no-warnings`; you'll miss deprecation notices before upgrades
- Publishing expensive payloads without checking `hasSubscribers`

TIPS

- Run CI with `--throw-deprecation` to catch deprecated APIs early
- Use `NODE_DEBUG=module` to see why an import resolves where it does
- Attach a diagnostic report to bug reports

SUMMARY

- `util.inspect` — full object output
- `debuglog` — opt-in debug logs
- `diagnostics_channel` — hooks for tracers
- `process.report` — JSON process snapshot

node:test Basics

THE BUILT-IN TEST RUNNER

`node:test` is a full test framework that ships with Node: test definitions, `describe` / `it` suites, hooks, subtests, skipping, mocking, snapshots, coverage, watch mode, and several reporters. Run every test file with `node --test`. For many projects it replaces Jest or Mocha, with no dependencies to install.

NOTES

- Each test file runs in its own process by default
- A test fails if it throws or its promise rejects
- Default files: `*.test.js`, `*-test.js`, `*_test.js`, `test/**`

WRITING TESTS

```
// math.test.js — run: node --test
import { test, describe, it, beforeEach } from 'node:test';
import assert from 'node:assert/strict';
import { add, divide, load } from './math.js';

test('adds numbers', () => {
  assert.equal(add(2, 3), 5);
});

describe('divide', () => {
  beforeEach(() => { /* fresh state before each test */ });
  it('divides', () => assert.equal(divide(6, 3), 2));
  it('throws on zero', () => assert.throws(() => divide(1, 0), /zero/));
  it('loads async data', async () => {
    assert.deepEqual(await load(), { ok: true });
  });
  it.skip('not ready yet', () => {});
});
```

CODE EXPLAINED

- → `test` / `it` — the same function under two names
- → `describe` — groups tests and scopes hooks
- → `assert/strict` — strict equality everywhere
- → `async` test — awaited; rejection = failure
- → `it.skip` — reported but not run; also `it.todo`

RUNNING TESTS

COMMAND	DOES
`node --test`	Run default patterns
`node --test 'src/**/*.test.js'`	Specific files
`node --test --watch`	Re-run on change
`--test-name-pattern=divide`	Filter by name
`--test-reporter=dot`	spec, tap, dot, junit

HOOKS

HOOK	RUNS
`before`	Once before the suite
`beforeEach`	Before every test
`afterEach`	After every test
`after`	Once after the suite

COMMON MISTAKES

- Forgetting `await` on `t.test()` or an async assertion; the test passes before checking
- Shared state between tests without `beforeEach`; order-dependent failures
- Leaving `it.only` in code; with `--test-only` everything else is skipped

TIPS

- Add `"test": "node --test"` to package.json scripts
- Use `--test-reporter=junit --test-reporter-destination=junit.xml` in CI
- Keep test files next to the code: `user.js` and `user.test.js`

SUMMARY

- `node --test` — run all tests
- `test / describe / it` — define tests
- `beforeEach` — reset state
- `--watch` — re-run on change

The assert Module

MAKING ASSERTIONS

`node:assert` checks values and throws an `AssertionError` with a readable diff when they don't match. Always import **`node:assert/strict`**: it uses === and strict deep comparison, so `1` never equals `'1'`. It works in any code, not just tests, which makes it handy for invariants.

NOTES

- Legacy (non-strict) mode uses `==`; avoid it
- Deep equality compares structure, not identity
- `throws` / `rejects` can match a class, regex, object, or function

COMMON ASSERTIONS

```
import assert from 'node:assert/strict';

assert.equal(1 + 1, 2);                          // ===
assert.notEqual('a', 'b');
assert.deepEqual({ a: [1, 2] }, { a: [1, 2] });  // structure
assert.notDeepEqual([1, 2], [2, 1]);
assert.ok(user.active);                          // truthy
assert.match('2026-10-02', /^\d{4}-\d{2}-\d{2}$/);

assert.throws(() => JSON.parse('{'), SyntaxError);
assert.throws(() => pay(-1), { name: 'RangeError', message: /negative/ });
await assert.rejects(getUser(999), { code: 'E_NOT_FOUND' });
await assert.doesNotReject(save());

assert.partialDeepStrictEqual(               // only the listed keys
  { id: 1, name: 'Ada', tags: [] },
  { name: 'Ada' },
);
assert.fail('should never get here');
```

CODE EXPLAINED

- → `deepEqual` (strict) — also compares prototypes
- → `ok` — fails on `0`, `''`, `null`, `undefined`
- → Object matcher — each listed property must match
- → `rejects` — takes a promise or async function; `await` it
- → `partialDeepStrictEqual` — subset match (newer API)

ASSERTION METHODS

METHOD	PASSES WHEN
`equal`	a === b
`deepEqual`	Same structure
`ok`	Truthy
`match`	Regex matches
`throws`	Function throws
`rejects`	Promise rejects
`strictEqual`	Alias of equal (strict)

ERROR MATCHERS

MATCHER	CHECKS
`SyntaxError`	instanceof
`/regex/`	Message matches
`{ code: 'X' }`	Properties match
`(err) => bool`	Custom check

STRICT VS LEGACY

CALL	STRICT / LEGACY
`equal(1, '1')`	fails / passes
`deepEqual([1], ['1'])`	fails / passes

COMMON MISTAKES

- Importing `node:assert` (legacy) and getting loose `==` comparisons
- `assert.rejects(...)` without `await`; the test ends before the check runs
- `assert.throws(fn())` instead of `assert.throws(() => fn())`; the error is thrown before assert sees it

TIPS

- Add a message as the last argument for clearer failures
- Use `t.assert` inside tests so assertions are tracked per test
- Use `assert.ok(cond, 'invariant')` in app code for impossible states

SUMMARY

- `assert/strict` — always import this
- `equal / deepEqual` — values and structures
- `throws / rejects` — expected errors
- `match` — regex checks

Mocking, Snapshots & Coverage

FAKES, SNAPSHOTS & COVERAGE

`node:test` includes **mocking**: spy on functions, replace object methods, and control **timers** so time-based code runs instantly. Mocks created through `t.mock` are restored when the test ends. **Snapshots** save a value to a file and compare future runs against it, and **coverage** reports which lines your tests executed.

NOTES

- `mock.fn()` records every call's args, result, and `this`
- Timer mocks also cover `Date` when enabled
- Module mocking needs `--experimental-test-module-mocks`

MOCKS, TIMERS & SNAPSHOTS

```
import { test, mock } from 'node:test';
import assert from 'node:assert/strict';

test('calls the API once', async (t) => {
  const f = t.mock.method(globalThis, 'fetch',
    async () => Response.json({ id: 1 }));        // fake response
  await loadUser(1);
  assert.equal(f.mock.callCount(), 1);
  assert.match(f.mock.calls[0].arguments[0], /\/users\/1$/);
});                                                // fetch restored here

test('debounce waits 300 ms', (t) => {
  t.mock.timers.enable({ apis: ['setTimeout'] });
  const fn = mock.fn();
  debounce(fn, 300)();
  t.mock.timers.tick(299); assert.equal(fn.mock.callCount(), 0);
  t.mock.timers.tick(1);   assert.equal(fn.mock.callCount(), 1);
});

test('renders card', (t) => {
  t.assert.snapshot(renderCard({ title: 'Hi' }));  // compares to stored
});
```

CODE EXPLAINED

- → `t.mock.method` — replaces a method for this test only
- → `mock.calls[i].arguments` — what each call received
- → `timers.tick(ms)` — advances fake time instantly
- → `mock.fn()` — a spy with no behavior
- → `snapshot` — first run writes the file, later runs compare

MOCK API

CALL	USE
`mock.fn(impl?)`	Spy / stub function
`mock.method(obj, name)`	Replace a method
`mock.getter / setter`	Fake properties
`mock.timers.enable()`	Fake time
`mock.module(spec)`	Fake a module
`mockImplementationOnce`	Next call only

COVERAGE & SNAPSHOTS

FLAG	DOES
`--experimental-test-coverage`	Coverage report
`--test-coverage-include`	Limit to globs
`--test-coverage-lines=80`	Fail under 80%
`--test-update-snapshots`	Rewrite snapshots
`--test-reporter=lcov`	For coverage tools

COMMON MISTAKES

- Using top-level `mock.method` instead of `t.mock`; it isn't restored between tests
- Snapshotting values with timestamps or random IDs; every run fails
- Treating 100% coverage as "tested"; covered lines can still lack assertions

TIPS

- Inject dependencies (pass `fetch` in) so you need fewer mocks
- Review snapshot diffs before running `--test-update-snapshots`
- Set a coverage threshold in CI to stop it slipping

SUMMARY

- `t.mock.method` — scoped stubs
- `mock.timers` — instant time travel
- `t.assert.snapshot` — store-and-compare
- `--experimental-test-coverage` — line coverage

Child Processes

RUNNING OTHER PROGRAMS

`node:child_process` starts other programs. `execFile` runs a command and buffers its output; `spawn` streams output for long-running or chatty commands; `exec` runs a string through a **shell** (pipes, globs, and also injection risk); `fork` starts another Node script with a message channel. Prefer the versions that **don't use a shell**.

NOTES

- Arguments as an array are never shell-interpreted
- `exec` / `execFile` buffer up to 1 MB of output by default
- On Windows, `.cmd` / `.bat` files need `shell: true`

EXECFILE, SPAWN, EXEC & FORK

```
import { spawn, execFile, exec, fork } from 'node:child_process';
import { promisify } from 'node:util';
const execFileP = promisify(execFile);

const { stdout } = await execFileP('git', ['rev-parse', 'HEAD']);
stdout.trim();                                  // 'a1b2c3d…'

const ff = spawn('ffmpeg', ['-i', 'in.mp4', 'out.webm'], { stdio: 'inherit' });
ff.on('close', (code) => console.log('exit', code));   // live output

const g = spawn('grep', ['ERROR']);
g.stdout.on('data', (d) => process.stdout.write(d));
g.stdin.end('ok\nERROR disk full\n');            // feed input, close stdin

await promisify(exec)('ls *.log | wc -l');       // shell: never with user input

const child = fork('./job.js');                   // Node + IPC channel
child.send({ jobId: 42 });
child.on('message', (m) => console.log('result', m));
```

CODE EXPLAINED

- → `execFile` + array — safe from shell injection
- → `stdio: 'inherit'` — child writes straight to your terminal
- → `stdin.end` — send input and signal EOF
- → `exec` — only for fixed strings you wrote yourself
- → `fork` — child calls `process.send()` to reply

WHICH FUNCTION

FUNCTION	SHELL / OUTPUT
`execFile`	No / buffered
`spawn`	No / streamed
`exec`	Yes / buffered
`fork`	No / IPC messages

STDIO OPTIONS

VALUE	MEANING
`'pipe'`	Access via child.stdout
`'inherit'`	Share parent's streams
`'ignore'`	Discard
`[in, out, err]`	Set each separately

USEFUL OPTIONS

OPTION	EFFECT
`cwd`	Working directory
`env`	Child environment
`timeout`	Kill after ms
`maxBuffer`	Output cap (exec)
`signal`	Abort the child

COMMON MISTAKES

- `exec('convert ' + filename)` with user input; a filename like `x; rm -rf ~` runs
- Not handling `'error'` on spawn; a missing binary emits ENOENT and crashes
- Ignoring the exit code; a failed command still "succeeds" if you don't check

TIPS

- Use `execa` for nicer promises, errors, and cross-platform behavior
- Pass `env: { ...process.env, LANG: 'C' }` for predictable command output
- Kill child processes on shutdown so they don't outlive the parent

SUMMARY

- `execFile` — run a command, get output
- `spawn` — stream long-running output
- `exec` — shell strings (careful)
- `fork` — Node child with messages

Worker Threads

PARALLEL JAVASCRIPT

`worker_threads` runs JavaScript on **additional threads** inside the same process, each with its own event loop and V8 isolate. Use them for **CPU-heavy** work (image processing, parsing, hashing, compression) so the main thread keeps answering requests. They don't help with I/O, which is already non-blocking.

NOTES

- Workers don't share variables; they exchange messages
- Messages are copied via structured clone
- Starting a worker costs tens of milliseconds; reuse them

OFFLOADING CPU WORK

```
// main.js
import { Worker } from 'node:worker_threads';

function hashInWorker(text) {
  return new Promise((resolve, reject) => {
    const url = new URL('./hash-worker.js', import.meta.url);
    const w = new Worker(url, { workerData: text });
    w.once('message', resolve);
    w.once('error', reject);
    w.once('exit', (code) => code && reject(new Error(`exit ${code}`)));
  });
}
console.log(await hashInWorker('big input'));   // main thread stayed free

// hash-worker.js
import { parentPort, workerData } from 'node:worker_threads';
import { createHash } from 'node:crypto';
const digest = createHash('sha256').update(workerData).digest('hex');
parentPort.postMessage(digest);
```

CODE EXPLAINED

- → `new URL(…, import.meta.url)` — worker path relative to this file
- → `workerData` — initial input, cloned into the worker
- → `once('message')` — first reply resolves the promise
- → `'exit'` with non-zero code — worker crashed
- → `parentPort` — the worker's line back to the main thread

KEY APIS

API	USE
`new Worker(file, opts)`	Start a thread
`workerData`	Initial input
`parentPort.postMessage`	Send to main
`worker.postMessage`	Send to worker
`worker.terminate()`	Stop it
`isMainThread`	Which side am I?

SHARING DATA

METHOD	COST
`postMessage(obj)`	Copied
`transferList`	Moved, zero-copy
`SharedArrayBuffer`	Shared memory
`Atomics`	Safe shared updates
`MessageChannel`	Worker ↔ worker

WORKERS OR NOT?

TASK	USE
`Heavy math, parsing`	Worker
`Image resize`	Worker (or sharp)
`DB / HTTP calls`	Plain async
`Separate programs`	child_process

COMMON MISTAKES

- Creating a new worker per request; startup cost exceeds the work, so use a pool
- Moving I/O into workers; it was never blocking the main thread
- Sending huge objects back and forth; cloning costs more than the computation

TIPS

- Use `piscina` for a ready-made worker pool
- Size pools with `os.availableParallelism()`
- Transfer `ArrayBuffer`s with `transferList` instead of copying

SUMMARY

- `Worker` — run JS on another thread
- `postMessage` — the only way to talk
- `SharedArrayBuffer` — opt-in shared memory
- `pool` — reuse workers

Cluster & Scaling

cluster.fork() · isPrimary · load balancing · sticky sessions · PM2 cluster mode · when to use workers instead

USING EVERY CORE

A Node process runs your JavaScript on one core. The `node:cluster` module **forks one process per core**, all listening on the **same port**; the primary process hands incoming connections to workers in turn. Today the same result usually comes from a process manager (PM2) or from running several containers behind a load balancer.

NOTES

- Workers are separate processes; they share no memory
- In-memory sessions or caches differ per worker
- Default scheduling is round-robin (except on Windows)

CLUSTER WITH AUTO-RESTART

```
import cluster from 'node:cluster';
import { availableParallelism } from 'node:os';
import { createServer } from 'node:http';

if (cluster.isPrimary) {
  const n = availableParallelism();
  for (let i = 0; i < n; i++) cluster.fork();      // one per core
  cluster.on('exit', (worker, code) => {
    console.log(`worker ${worker.process.pid} exited (${code})`);
    if (code !== 0) cluster.fork();                // replace crashed workers
  });
} else {
  createServer((req, res) => res.end(`pid ${process.pid}\n`))
    .listen(3000);                                 // all share port 3000
}
```

CODE EXPLAINED

- → `isPrimary` — the same file runs in every process
- → `fork()` — starts a worker running this file
- → `'exit'` — restart only on failure, not on shutdown
- → `listen(3000)` — the primary owns the socket

SCALING OPTIONS

OPTION	BEST FOR
`cluster`	Single VM, no extra tools
`PM2 -i max`	Single VM, ops features
`Containers + LB`	Kubernetes, cloud
`Serverless`	Spiky traffic

CLUSTER API

API	USE
`isPrimary / isWorker`	Which role
`fork(env?)`	Start a worker
`worker.send(msg)`	Message a worker
`worker.disconnect()`	Graceful stop
`cluster.workers`	Active workers

PM2 EQUIVALENT

```
pm2 start app.js -i max
pm2 reload app
pm2 logs
pm2 save && pm2 startup
```

COMMON MISTAKES

- Storing sessions or rate-limit counters in memory; each worker sees a different subset of users
- Running cluster inside Kubernetes pods that already scale horizontally; it fights CPU limits
- Restarting workers in a tight loop when they crash at startup

TIPS

- Put shared state in Redis or a database
- Prefer one process per container and scale containers
- Back off restarts if workers die within seconds of starting

SUMMARY

- `cluster.fork()` — process per core
- `shared port` — primary balances connections
- `PM2` — cluster plus ops tools
- `external state` — required for scaling

Measuring Performance

FINDING WHAT'S SLOW

Measure before optimizing. `performance.now()` and marks/measures time specific code. **Event loop delay** tells you whether the main thread is blocked, the most common cause of a slow Node server. For "where does the time go?", record a **CPU profile** and read it as a flame graph in Chrome DevTools.

NOTES

- `performance.now()` is monotonic; `Date.now()` can jump
- Event loop delay above ~100 ms means users notice
- Profile production-like builds; dev mode skews results

TIMING & LOOP HEALTH

```
import { performance, monitorEventLoopDelay } from 'node:perf_hooks';

const t0 = performance.now();
await work();
console.log(`${(performance.now() - t0).toFixed(1)} ms`);   // 41.7 ms

performance.mark('start'); await work(); performance.mark('end');
performance.measure('work', 'start', 'end').duration;      // 41.7

const h = monitorEventLoopDelay({ resolution: 20 });
h.enable();
setInterval(() => {
  console.log('p99 delay', (h.percentile(99) / 1e6).toFixed(1), 'ms');
  h.reset();
}, 10_000).unref();

performance.eventLoopUtilization().utilization;   // 0.37 (37% busy)
const timed = performance.timerify(slowFn);       // records each call
```

CODE EXPLAINED

- → `mark / measure` — named spans, visible in profilers
- → `monitorEventLoopDelay` — histogram in nanoseconds
- → `/ 1e6` — nanoseconds → milliseconds
- → `eventLoopUtilization` — near 1 means CPU-bound
- → `timerify` — wraps a function to time every call

PROFILING FLAGS

FLAG	OUTPUT
`--cpu-prof`	.cpuprofile on exit
`--cpu-prof-dir=dir`	Where to save
`--heap-prof`	Allocation profile
`--prof`	V8 tick log

PROFILE A SCRIPT

```
node --cpu-prof app.js
# open the .cpuprofile in
# DevTools → Performance
npx 0x app.js   # flame graph
```

HEALTH SIGNALS

METRIC	WATCH FOR
`ELU`	> 0.8 sustained
`Loop delay p99`	> 100 ms
`Heap used`	Steady growth
`Response time`	p95 / p99

COMMON MISTAKES

- Optimizing on a hunch; most time is usually in one or two functions
- Benchmarking a single run; JIT warm-up dominates, so run many iterations
- Measuring with `Date.now()`; millisecond resolution and clock jumps

TIPS

- Export loop delay and ELU as metrics (Sheet 58)
- Use `autocannon` to load-test an HTTP endpoint
- Profile under realistic load, not with one request

SUMMARY

- `performance.now` — precise timing
- `monitorEventLoopDelay` — blocked-loop detector
- `ELU` — how busy the main thread is
- `--cpu-prof` — flame-graph profiles

Memory & Garbage Collection

WHERE MEMORY GOES

V8 stores JavaScript objects in a **garbage-collected heap** with a size limit; the process also uses memory outside it (Buffers, native code). Memory that keeps growing under steady load is a **leak**: something still references objects you no longer need. **Heap snapshots** show what's holding them. `WeakMap` and `WeakRef` let you reference objects without keeping them alive.

NOTES

- `rss` = everything the OS gave the process
- Exceeding the heap limit kills the process ("heap out of memory")
- GC pauses stop JavaScript briefly

MEASURING & AVOIDING LEAKS

```
import v8 from 'node:v8';

process.memoryUsage();
// { rss: 81M, heapTotal: 22M, heapUsed: 15M, external: 2M, arrayBuffers: 1M }
v8.getHeapStatistics().heap_size_limit / 1024 ** 2;   // max heap in MB
v8.writeHeapSnapshot();            // 'Heap.20261002….heapsnapshot'

const cache = new Map();           // ✖ grows forever without eviction
const meta = new WeakMap();        // ✔ entry freed with its key object
meta.set(req, { start: Date.now() });

const ref = new WeakRef(bigObject);       // doesn't keep it alive
ref.deref()?.render();                    // undefined once collected
const reg = new FinalizationRegistry((id) => releaseNative(id));
reg.register(handle, handle.id);          // callback after GC (no guarantee)
```

CODE EXPLAINED

- → `heapUsed` — live JS objects; watch this trend
- → `external` — Buffers and native allocations
- → `writeHeapSnapshot` — open in DevTools → Memory
- → `WeakMap` — keys must be objects
- → `FinalizationRegistry` — cleanup hint, never for critical logic

COMMON LEAKS

CAUSE	FIX
`Unbounded Map cache`	LRU with max size
`Listeners never removed`	off() / once()
`Timers never cleared`	clearInterval
`Closures in long-lived arrays`	Remove when done
`Global request arrays`	Don't store per-request data

MEMORYUSAGE()

FIELD	MEANS
`rss`	Total resident memory
`heapTotal`	Heap reserved
`heapUsed`	Heap in use
`external`	C++ / Buffer memory

MEMORY FLAGS

```
node --max-old-space-size=4096 app.js
node --heapsnapshot-signal=SIGUSR2 app.js
node --heapsnapshot-near-heap-limit=2 app.js
```

COMMON MISTAKES

- Setting `--max-old-space-size` above the container's memory limit; the OS kills it first
- Comparing one heap snapshot; take two or three over time and diff them
- Using a `Map` as a cache without a size limit

TIPS

- Set the heap limit to ~75% of the container's memory
- Use `lru-cache` for bounded in-memory caches
- Compare snapshots with the "Comparison" view to find what grew

SUMMARY

- `memoryUsage()` — current footprint
- `heap snapshot` — who holds what
- `WeakMap / WeakRef` — non-owning references
- `--max-old-space-size` — heap limit in MB

Permission Model

RESTRICTING WHAT A PROCESS CAN DO

Normally Node code can read any file, spawn any program, and open any connection the OS user can. The **permission model** turns that around: with `--permission`, file access, child processes, workers, native addons, and WASI are **denied** until you grant them with `--allow-*` flags. A blocked call throws `ERR_ACCESS_DENIED`. It limits damage from bugs and compromised dependencies; it isn't a sandbox for code you know is malicious.

NOTES

- Paths can be files, folders (trailing `/`), or `*`
- `--allow-net` (network restriction) is only in newer releases
- Permissions apply to the whole process, including dependencies

GRANTING PERMISSIONS

```
node --permission app.js                      # everything below is denied
node --permission --allow-fs-read=./data/ --allow-fs-read=./config.json app.js
node --permission --allow-fs-read=* --allow-fs-write=/tmp/app/ app.js
node --permission --allow-child-process app.js
node --permission --allow-worker app.js
node --permission --allow-addons --allow-wasi app.js
node --permission --allow-net app.js          # newer releases only
```

CODE EXPLAINED

→ No `--allow-*` — the app can't read its own data files

→ Repeat a flag to allow several paths

→ Trailing `/` — grants the whole folder

→ `*` — all paths for that operation

→ The entry file itself is always readable

FLAGS

FLAG	GRANTS
`--allow-fs-read=p`	Read files
`--allow-fs-write=p`	Write files
`--allow-child-process`	spawn / exec
`--allow-worker`	worker_threads
`--allow-addons`	Native .node modules
`--allow-wasi`	WASI

PROCESS.PERMISSION

CALL	RESULT
`has('fs.read')`	Any read?
`has('fs.read', p)`	This path?
`has('child')`	Can spawn?

WHAT CODE SEES

```
process.permission
  .has('fs.write');   // false
await writeFile('x', 'a');
// throws:
// ERR_ACCESS_DENIED
```

COMMON MISTAKES

- Granting `--allow-fs-read=*` "to make it work"; you lose most of the benefit
- Assuming it blocks network access on releases without `--allow-net`
- Treating it as a sandbox for untrusted plugins; use a separate process or container

TIPS

- Start with nothing, run your tests, and add flags for each `ERR_ACCESS_DENIED`
- Put the flags in the start script so production and CI match
- Combine with a non-root user and read-only container filesystem

SUMMARY

- `--permission` — deny by default
- `--allow-fs-read / write` — grant paths
- `ERR_ACCESS_DENIED` — what blocked calls throw
- `process.permission.has` — check at runtime

Crypto

HASHING, TOKENS & ENCRYPTION

`node:crypto` covers secure **random values**, **hashes**, **HMAC** signatures, **password hashing**, and **encryption**. The global `crypto` also exposes the browser-standard Web Crypto API (`crypto.subtle`), which is the better choice for code that must run in other runtimes. Never use `Math.random()` for anything security-related.

NOTES

- Hashes are one-way; encryption is reversible with the key
- Fast hashes (SHA-256) are wrong for passwords; use scrypt or argon2
- AES-GCM needs a unique IV for every message

COMMON CRYPTO TASKS

```
import { randomUUID, randomBytes, randomInt, createHash, createHmac,
  scrypt, timingSafeEqual } from 'node:crypto';
import { promisify } from 'node:util';

randomUUID();                                  // 'c0a8f2e4-…' (v4)
randomBytes(32).toString('base64url');         // session / API token
randomInt(1, 7);                               // 1-6, unbiased
createHash('sha256').update('hello').digest('hex');   // '2cf24dba5f…'
createHmac('sha256', secret).update(body).digest('hex');   // signature

const salt = randomBytes(16);
const hash = await promisify(scrypt)(password, salt, 64);  // store salt+hash
timingSafeEqual(hash, storedHash);             // constant-time compare

const key = await crypto.subtle.generateKey(
  { name: 'AES-GCM', length: 256 }, true, ['encrypt', 'decrypt']);
const iv = crypto.getRandomValues(new Uint8Array(12));
const sealed = await crypto.subtle.encrypt({ name: 'AES-GCM', iv }, key, data);
```

CODE EXPLAINED

- → `base64url` — URL-safe token text
- → `randomInt` — avoids the bias of `% n`
- → HMAC — verifies webhooks were sent by someone with the secret
- → scrypt — deliberately slow; salt defeats precomputed tables
- → `timingSafeEqual` — `===` leaks timing information

PICK THE TOOL

TASK	USE
IDs	randomUUID()
Tokens	randomBytes(32)
File checksum	SHA-256
Webhook signature	HMAC-SHA256
Passwords	scrypt / argon2
Encrypt data	AES-256-GCM

AVOID

DON'T	WHY
Math.random	Predictable
MD5 / SHA-1	Broken for security
SHA-256 for passwords	Too fast to brute-force
Reusing an IV	Breaks GCM
Your own crypto	Use vetted primitives

COMMON MISTAKES

- Comparing signatures with `===`; use `timingSafeEqual` on equal-length Buffers
- Storing a password hash without its salt; you can't verify it later
- Hardcoding encryption keys in source; load them from a secret manager

TIPS

- Use `crypto.hash('sha256', data)` for quick one-shot hashes
- Store password hashes as `salt:hash` in hex or base64
- Prefer a maintained library (`jose`) for JWTs instead of hand-rolling

SUMMARY

- `randomUUID / randomBytes` — secure randomness
- `createHash / createHmac` — digests and signatures
- `scrypt` — password hashing
- `crypto.subtle` — Web Crypto encryption

Supply Chain & Hardening

YOUR DEPENDENCIES ARE YOUR CODE

A typical Node app runs hundreds of packages written by strangers, and any of them can run code at **install time** (install scripts) or **run time**. Supply-chain attacks (typosquatted names, hijacked maintainer accounts, malicious updates) target exactly that. Defend with a committed **lockfile**, few dependencies, audits, and an install process that doesn't run arbitrary scripts.

NOTES

- `npm ci` installs exactly what the lockfile says
- `postinstall` scripts run with your user's permissions
- Patch releases can contain anything; ranges pull them in

AUDIT & HARDEN

```
npm audit                              # known vulnerabilities
npm audit --omit=dev                   # only what ships to production
npm audit signatures                   # verify registry signatures
npm ci --ignore-scripts                # install without install scripts
npm ls --all | wc -l                   # how many packages you really ship
npm explain minimist                   # why a package is in the tree
npm view some-pkg maintainers time.modified   # vet before adding
npm publish --provenance               # prove your package came from CI
npm outdated                           # stale versions
```

CODE EXPLAINED

- → `--omit=dev` — dev tool advisories rarely matter in production
- → `signatures` — detects tampered packages
- → `--ignore-scripts` — then allow-list packages that need them
- → `ls --all` — the real dependency count is often 10× direct deps
- → `--provenance` — links the package to its source commit

THREATS

ATTACK	DEFENSE
Typosquatting	Double-check names
Install script malware	--ignore-scripts
Hijacked maintainer	Lockfile, slow upgrades
Malicious update	Review diffs, pin
Protestware	Audit + vendor critical deps

RUNTIME HARDENING

MEASURE	EFFECT
Non-root user	Limits OS damage
--permission	Limits fs / spawn
NODE_ENV=production	Disables dev paths
Secrets in env/vault	Not in the repo
Security headers	helmet / @fastify/helmet

CI CHECKLIST

STEP
npm ci, not install
npm audit --omit=dev
Dependabot / Renovate
Fail on lockfile drift

COMMON MISTAKES

- Running `npm install some-package` from a blog post without checking the exact name
- Auto-merging every dependency update; a compromised release ships straight to production
- Ignoring `npm audit` entirely because it's noisy; filter it instead

TIPS

- Before adding a dependency, ask whether 20 lines of your own code would do
- Group upgrades weekly with Renovate and review the changelogs
- Subscribe to Node's security release announcements

SUMMARY

- `lockfile + npm ci` — reproducible installs
- `npm audit` — known vulnerabilities
- `--ignore-scripts` — no install-time code
- `fewer deps` — smaller attack surface

The util Module

SMALL HELPERS YOU ALREADY HAVE

`node:util` is a grab bag of helpers used throughout Node itself. Several have their own sheets (`promisify` 14, `parseArgs` 22, `styleText` 24, `inspect` / `debuglog` 46); this sheet collects the rest. Check here before installing a utility package.

NOTES

- `util.types` checks work across realms (workers, vm)
- `util.deprecate` warns once per process
- Many old `util.is*` helpers are removed; use `typeof` / `Array.isArray`

EVERYDAY UTIL FUNCTIONS

```
import util from 'node:util';

util.format('%s is %d', 'Ada', 36);              // 'Ada is 36'
util.isDeepStrictEqual({ a: [1] }, { a: [1] });  // true
util.types.isPromise(Promise.resolve());         // true
util.types.isDate(new Date());                   // true
util.types.isNativeError(new TypeError());       // true
util.getSystemErrorName(-2);                     // 'ENOENT'
util.stripVTControlCharacters('\x1b[31mred\x1b[0m');   // 'red'
util.toUSVString('bad \ud800 surrogate');        // replaces lone surrogates

export const oldApi = util.deprecate(
  newApi, 'oldApi() is deprecated; use newApi()', 'DEP_APP_001');

await util.aborted(signal, resource);   // resolves when signal aborts
const mime = new util.MIMEType('text/html; charset=utf-8');
mime.essence;                           // 'text/html'
mime.params.get('charset');             // 'utf-8'
```

CODE EXPLAINED

- → `format` — same placeholders as `console.log`
- → `isDeepStrictEqual` — the comparison `assert` uses
- → `getSystemErrorName` — errno number → code
- → `stripVTControlCharacters` — remove ANSI colors
- → `deprecate` — wrap old functions to warn callers

UTIL ELSEWHERE

FUNCTION	SHEET
`promisify / callbackify`	14
`parseArgs`	22
`styleText`	24
`inspect / debuglog`	46
`parseEnv`	20

UTIL.TYPES CHECKS

FUNCTION	TRUE FOR
`isPromise`	Promises
`isRegExp`	RegExps
`isAsyncFunction`	async functions
`isTypedArray`	Uint8Array etc.
`isProxy`	Proxies

PREFER INSTEAD

OLD	USE
`util.isArray`	Array.isArray
`util.inherits`	class extends
`promisify(setTimeout)`	timers/promises

COMMON MISTAKES

- Using removed helpers like `util.isString`; they throw on current versions
- Comparing objects with `JSON.stringify`; key order differs, use `isDeepStrictEqual`
- Formatting user strings with `util.format` as a template; `%` in input gets interpreted

TIPS

- Wrap legacy exports with `deprecate` before removing them
- Use `util.types` checks for values that come from other realms
- Run `node -p "Object.keys(require('node:util'))"` to see everything

SUMMARY

- `format` — printf-style strings
- `isDeepStrictEqual` — deep compare
- `types` — reliable type checks
- `deprecate` — warn on old APIs

AsyncLocalStorage

REQUEST CONTEXT WITHOUT PARAMETERS

`AsyncLocalStorage` stores a value that follows **one logical operation** through every `await`, callback, and timer it triggers, like thread-local storage for async code. Start a context with `run(store, fn)`; anything that runs inside, however deep, reads it with `getStore()`. The classic use is a **request ID** in every log line without passing it through every function.

NOTES

- Each concurrent request gets its own store
- `getStore()` returns `undefined` outside a `run()`
- Many tracing and logging libraries use it internally

A REQUEST ID IN EVERY LOG

```
import { AsyncLocalStorage } from 'node:async_hooks';
import { randomUUID } from 'node:crypto';
import { createServer } from 'node:http';

const ctx = new AsyncLocalStorage();

function log(msg) {
  const id = ctx.getStore()?.requestId ?? '-';
  console.log(`[${id}] ${msg}`);
}

async function loadUser(id) {
  log(`loading user ${id}`);                    // no requestId parameter
  await new Promise((r) => setTimeout(r, 10));
  log('loaded');                                // still the same request
}

createServer((req, res) => {
  const requestId = req.headers['x-request-id'] ?? randomUUID();
  ctx.run({ requestId }, async () => {
    await loadUser(7);                          // [3f2c…] loading user 7
    res.end('ok');
  });
}).listen(3000);
```

CODE EXPLAINED

- → One instance — create at module level, share it
- → `run(store, fn)` — store is visible inside `fn` and its async work
- → `?.` — handles code that runs outside a request
- → `x-request-id` — reuse the ID from a proxy if present

API

MEMBER	DOES
`run(store, fn)`	Start a context
`getStore()`	Current store
`exit(fn)`	Run without context
`enterWith(store)`	Set for the rest of sync code
`AsyncLocalStorage.bind(fn)`	Capture current context
`AsyncLocalStorage.snapshot()`	Capture for later

COMMON USES

USE	STORE
Log correlation	requestId
Auth	current user
Transactions	DB client
Tracing	span

COMMON MISTAKES

- Creating a new `AsyncLocalStorage` per request; nothing else can see it
- Storing a primitive and expecting to update it; store an object and mutate its fields
- Using `enterWith` in shared code; context leaks into unrelated work

TIPS

- Wrap it in a tiny module: `getRequestId()`, `withContext()`
- Add the store to logger child bindings automatically (Sheet 65)
- Keep stores small; they live as long as the async work

SUMMARY

- `AsyncLocalStorage` — async-safe context
- `run(store, fn)` — begin
- `getStore()` — read anywhere inside
- `requestId` — the classic use

node:sqlite

A DATABASE BUILT IN

`node:sqlite` embeds **SQLite** in Node: a full SQL database in a single file, with no server and no package to install. The API is **synchronous**, which suits CLIs, tests, scripts, desktop apps, and small services (SQLite queries usually take microseconds). Use **prepared statements** with parameters for every query that includes data.

NOTES

- Only importable with the `node:` prefix
- Check the docs for its current stability level
- `':memory:'` creates a throwaway in-memory database

CREATE, INSERT, QUERY, TRANSACT

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync('notes.db');        // or ':memory:'
db.exec(`
  CREATE TABLE IF NOT EXISTS notes (
    id INTEGER PRIMARY KEY, title TEXT NOT NULL, done INTEGER DEFAULT 0
  ) STRICT;
`);

const insert = db.prepare('INSERT INTO notes (title) VALUES (?)');
insert.run('Buy milk');          // { changes: 1, lastInsertRowid: 1 }

const byId = db.prepare('SELECT * FROM notes WHERE id = :id');
byId.get({ id: 1 });             // { id: 1, title: 'Buy milk', done: 0 }
db.prepare('SELECT * FROM notes WHERE done = ?').all(0);   // all rows

db.exec('BEGIN');
try { for (const t of titles) insert.run(t); db.exec('COMMIT'); }
catch (err) { db.exec('ROLLBACK'); throw err; }
db.close();
```

CODE EXPLAINED

- → `exec` — runs SQL with no parameters or results
- → `STRICT` — SQLite enforces column types
- → `?` / `:id` — positional and named parameters
- → `get` — first row or `undefined`; `all` — array
- → Transactions — batch inserts are far faster inside one

STATEMENT METHODS

METHOD	RETURNS
`run(...params)`	{ changes, lastInsertRowid }
`get(...params)`	One row / undefined
`all(...params)`	Array of rows
`iterate(...params)`	Row iterator
`columns()`	Column info

DATABASE METHODS

METHOD	DOES
`exec(sql)`	Run SQL text
`prepare(sql)`	Compile a statement
`close()`	Close the file
`function(name, fn)`	Add a SQL function

GOOD FIT?

USE	FIT
`CLI / desktop data`	✔
`Tests, prototypes`	✔
`Read-heavy small site`	✔
`Many writers, many servers`	✖

COMMON MISTAKES

- Building SQL with template strings and user input; that's SQL injection, so use parameters
- Inserting thousands of rows without a transaction; each insert syncs to disk
- Calling it from a busy server's request path for slow queries; it blocks the loop

TIPS

- Run `PRAGMA journal_mode = WAL;` for better read/write concurrency
- Reuse prepared statements instead of preparing in loops
- Use `:memory:` databases for fast, isolated tests

SUMMARY

- `DatabaseSync` — open a SQLite file
- `prepare + run/get/all` — parameterized queries
- `BEGIN / COMMIT` — transactions
- `synchronous` — fast, but blocks

Express Basics

THE MOST COMMON NODE FRAMEWORK

Express wraps `node:http` with routing, request helpers, and middleware. Routes match a **method** and **path**; path segments like `:id` land in `req.params`, query strings in `req.query`, and parsed bodies in `req.body`. **Express 5** (current) forwards errors from `async` handlers automatically, so a rejected promise reaches your error handler instead of hanging the request.

NOTES

- `req.body` is `undefined` unless body-parsing middleware runs
- `res.json` sets `Content-Type` and stringifies
- Routes are checked top to bottom; first match wins

ROUTES & RESPONSES

```
import express from 'express';                    // npm i express

const app = express();
app.use(express.json());                          // parse JSON bodies

app.get('/', (req, res) => res.send('Hello'));
app.get('/notes/:id', async (req, res) => {
  const note = await db.find(req.params.id);      // '42' (a string)
  if (!note) return res.status(404).json({ error: 'Not found' });
  res.json(note);
});
app.get('/search', (req, res) => res.json({ q: req.query.q }));   // ?q=node
app.post('/notes', async (req, res) => {
  const note = await db.create(req.body);
  res.status(201).location(`/notes/${note.id}`).json(note);
});
app.delete('/notes/:id', async (req, res) => {
  await db.remove(req.params.id);
  res.sendStatus(204);
});

app.listen(3000, () => console.log('http://localhost:3000'));
```

CODE EXPLAINED

- → `express.json()` — fills `req.body` for JSON requests
- → `:id` — path parameter, always a string
- → `return res…` — stop after responding
- → `location()` — sets the `Location` header for 201s
- → `sendStatus(204)` — status with no body

REQUEST

MEMBER	VALUE
`req.params`	Path segments
`req.query`	Query string object
`req.body`	Parsed body
`req.get('h')`	A header
`req.ip`	Client IP

RESPONSE

METHOD	DOES
`res.json(obj)`	Send JSON
`res.send(x)`	String / Buffer
`res.status(n)`	Set status (chainable)
`res.redirect(url)`	302 redirect
`res.sendFile(p)`	Stream a file

EXPRESS 5 CHANGES

CHANGE	DETAIL
`Async errors`	Forwarded to next(err)
`Wildcards`	'/*splat', not '*'
`Optional`	'/:id{/:slug}'
`req.query`	Read-only getter

COMMON MISTAKES

- Forgetting `express.json()`; `req.body` is `undefined`
- Sending a response twice (no `return`); throws `ERR_HTTP_HEADERS_SENT`
- Express 4 habits on v5: `'*'` route paths throw at startup

TIPS

- Validate `req.params` / `req.body` with a schema before use
- Set `app.set('trust proxy', 1)` behind one proxy so `req.ip` is correct
- Split routes into Routers by resource (Sheet 62)

SUMMARY

- `app.get / post / …` — define routes
- `req.params / query / body` — inputs
- `res.status().json()` — responses
- `Express 5` — async errors handled

Express Middleware & Routers

MIDDLEWARE & ROUTERS

Middleware is a function `(req, res, next)` that runs before your routes: it can read or change the request, end the response, or call `next()` to pass control on. Order matters: middleware runs in the order you register it. **Routers** group related routes and middleware and mount them at a prefix. An **error handler** has four parameters and goes last.

NOTES

- Not calling `next()` (or responding) hangs the request
- `next(err)` skips to the error handler
- `app.use(path, …)` matches that prefix

LOGGER, ROUTER, 404 & ERROR HANDLER

```
import express from 'express';
const app = express();

app.use((req, res, next) => {                    // request logger
  const t0 = performance.now();
  res.on('finish', () => console.log(req.method, req.originalUrl,
    res.statusCode, `${Math.round(performance.now() - t0)}ms`));
  next();
});
app.use(express.static('public'));               // files from ./public
app.use('/api', express.json());

const notes = express.Router();
notes.get('/', listNotes);
notes.post('/', requireAuth, createNote);        // per-route middleware
app.use('/api/notes', notes);                    // mount at a prefix

app.use((req, res) => res.status(404).json({ error: 'Not found' }));
app.use((err, req, res, next) => {               // error handler: 4 args
  req.log?.error(err);
  res.status(err.status ?? 500).json({ error: 'Something went wrong' });
});
```

CODE EXPLAINED

- → `'finish'` — fires after the response is sent
- → `static` — serves files and stops if one matches
- → `requireAuth` — middleware for just one route
- → `originalUrl` — full URL even inside a mounted router
- → Four params — Express identifies error handlers by arity

MIDDLEWARE KINDS

KIND	REGISTERED WITH
App-level	app.use(fn)
Path-level	app.use('/api', fn)
Route-level	app.get(p, mw, handler)
Router-level	router.use(fn)
Error	app.use((err, …) => {})

COMMON PACKAGES

PACKAGE	DOES
helmet	Security headers
cors	CORS headers
express-rate-limit	Throttling
cookie-parser	req.cookies
multer	File uploads
pino-http	Request logging

TYPICAL ORDER

STEP
Logging, helmet
CORS, body parsing
Auth
Routes
404 → error handler

COMMON MISTAKES

- Registering the error handler before the routes; it never sees their errors
- Leaking `err.message` or stack to clients in production
- Parsing JSON globally when only `/api` needs it

TIPS

- Write `requireAuth` once and attach it per Router
- Send a generic message to clients, log the full error
- Use `express.Router({ mergeParams: true })` for nested params

SUMMARY

- `(req, res, next)` — middleware signature
- `next(err)` — jump to error handling
- `Router` — group and mount routes
- `(err, req, res, next)` — error handler, last

Fastify

A FASTER, SCHEMA-DRIVEN FRAMEWORK

Fastify is a modern alternative to Express focused on speed and structure. Handlers can simply **return** a value, routes can declare a **JSON Schema** that validates input and speeds up output serialization, and everything is organized as **plugins** with encapsulated scope. It ships with **pino** logging built in.

NOTES

- Schema validation errors return 400 automatically
- Response schemas drop properties not listed (no accidental leaks)
- Use `fastify-type-provider-zod` / TypeBox for typed schemas

ROUTES, SCHEMAS & PLUGINS

```
import Fastify from 'fastify';                    // npm i fastify

const app = Fastify({ logger: true });

app.get('/notes/:id', async (req, reply) => {
  const note = await db.find(req.params.id);
  if (!note) return reply.code(404).send({ error: 'Not found' });
  return note;                                    // sent as JSON
});

app.post('/notes', {
  schema: {
    body: {
      type: 'object', required: ['title'],
      properties: { title: { type: 'string', minLength: 1 } },
    },
  },
}, async (req, reply) => reply.code(201).send(await db.create(req.body)));

app.register(import('./routes/users.js'), { prefix: '/users' });
app.addHook('onRequest', async (req) => { /* auth check */ });

await app.listen({ port: 3000, host: '0.0.0.0' });
```

CODE EXPLAINED

- → `logger: true` — request logs with IDs, via pino
- → `return note` — no `res.json` needed
- → `schema.body` — rejected requests never reach the handler
- → `register` — plugins get their own encapsulated scope
- → `host: '0.0.0.0'` — default is localhost only

FASTIFY VS EXPRESS

FEATURE	FASTIFY / EXPRESS
Return values	Yes / No
Validation	Built in / Add a lib
Logging	Built in / Add a lib
Async errors	Yes / Yes (v5)
Ecosystem	Large / Largest

LIFECYCLE HOOKS

HOOK	RUNS
onRequest	First (auth)
preValidation	Before schema check
preHandler	After validation
onSend	Before sending
onError	On errors

OFFICIAL PLUGINS

PLUGIN	DOES
@fastify/cors	CORS
@fastify/helmet	Security headers
@fastify/rate-limit	Throttling
@fastify/static	Files

COMMON MISTAKES

- Calling `reply.send()` and also returning a value; Fastify warns about a double reply
- Expecting a decorator or hook from one plugin to be visible in a sibling; wrap with `fastify-plugin` to share
- Listening without `host` in Docker; the container isn't reachable

TIPS

- Add response schemas to every public route
- Use `app.inject()` to test routes without opening a port
- Generate OpenAPI docs from schemas with `@fastify/swagger`

SUMMARY

- `return value` — the response body
- `schema` — validation + fast JSON output
- `register` — plugins and encapsulation
- `hooks` — request lifecycle

Databases from Node

pg Pool · parameterized queries · mysql2 · Prisma · Drizzle · connection pooling · transactions

TALKING TO DATABASES

Node apps reach SQL databases through a **driver** (`pg` for PostgreSQL, `mysql2` for MySQL), usually behind a **connection pool** that reuses a fixed number of connections. Always use **parameterized queries**. **ORMs** and query builders (Prisma, Drizzle, Kysely) add types and migrations on top of the same drivers.

NOTES

- Create one pool per process and reuse it
- A transaction must run on a single client
- `$1, $2` (pg) and `?` (mysql2) are parameter placeholders

POOLING, PARAMETERS & TRANSACTIONS

```
import pg from 'pg';                                  // npm i pg
const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL });

const { rows } = await pool.query(
  'SELECT id, title FROM notes WHERE owner_id = $1 AND done = $2',
  [userId, false],
);

const client = await pool.connect();                  // one client per transaction
try {
  await client.query('BEGIN');
  await client.query('UPDATE acct SET bal = bal - $1 WHERE id = $2', [50, a]);
  await client.query('UPDATE acct SET bal = bal + $1 WHERE id = $2', [50, b]);
  await client.query('COMMIT');
} catch (err) {
  await client.query('ROLLBACK');
  throw err;
} finally {
  client.release();                                   // back to the pool
}
```

CODE EXPLAINED

- → `Pool` — opens connections lazily up to `max` (default 10)
- → `$1, $2` — values sent separately; injection-safe
- → `pool.connect()` — check out one client
- → `ROLLBACK` — undo both updates on any error
- → `release()` — in `finally`, or the pool runs dry

DRIVERS

DATABASE	PACKAGE
PostgreSQL	pg / postgres
MySQL	mysql2
SQLite	node:sqlite / better-sqlite3
MongoDB	mongodb / mongoose
Redis	redis / ioredis

ORMS & BUILDERS

TOOL	STYLE
Prisma	Schema file, generated client
Drizzle	TS schema, SQL-like API
Kysely	Typed query builder
Sequelize / TypeORM	Classic ORM

POOL SETTINGS

OPTION	EFFECT
max	Max connections
idleTimeoutMillis	Close idle clients
connectionTimeoutMillis	Wait for a client

COMMON MISTAKES

- Building `WHERE id = ${id}` from user input; that's SQL injection, so use parameters
- Running `BEGIN` via `pool.query`; each call may use a different connection
- Forgetting `release()`; the app freezes once all clients are checked out

TIPS

- Size `max` so instances × max stays under the DB's connection limit
- Close the pool on shutdown: `await pool.end()` (Sheet 19)
- Use PgBouncer or a serverless driver for many short-lived instances

SUMMARY

- `Pool` — reuse connections
- `$1 / ?` — always parameterize
- `connect + BEGIN/COMMIT` — transactions
- `ORMs` — types and migrations on top

Logging

STRUCTURED LOGS WITH PINO

Production logs should be **structured** (one JSON object per line) so log tools can search and aggregate them. **pino** is the standard Node logger: very fast, JSON by default, with **levels**, **child loggers** that add fields to every line, and **redaction** of secrets. In development, pipe its output through `pino-pretty` for readable lines.

NOTES

- Fields first, message second: `log.info({ id }, 'msg')`
- Pass errors as `{ err }` to get the stack serialized
- Fastify uses pino; Express uses `pino-http`

LOGGER SETUP & USAGE

```
import pino from 'pino';                          // npm i pino

const log = pino({
  level: process.env.LOG_LEVEL ?? 'info',
  redact: ['req.headers.authorization', '*.password'],
});

log.info('server started');
// {"level":30,"time":1759400000000,"pid":812,"msg":"server started"}
log.info({ userId: 7, ms: 42 }, 'login ok');
log.error({ err }, 'payment failed');             // includes err.stack
log.debug('hidden unless LOG_LEVEL=debug');

const reqLog = log.child({ requestId: 'a1b2' });  // adds requestId to each line
reqLog.warn({ sql }, 'slow query');

// dev only:  node app.js | npx pino-pretty
```

CODE EXPLAINED

- → `level` — lines below it are skipped cheaply
- → `redact` — paths replaced with `[Redacted]`
- → `{ err }` — the `err` key triggers the error serializer
- → `child` — per-request or per-module context
- → `pino-pretty` — human-readable, never in production

LEVELS

LEVEL	NUMBER
trace	10
debug	20
info	30
warn	40
error	50
fatal	60

WHAT TO LOG

DO	DON'T
Request IDs	Passwords, tokens
Durations	Full request bodies
Error stacks	Personal data
State changes	Every loop iteration

SHIPPING LOGS

OPTION	HOW
stdout	Let the platform collect
pino.transport	Worker-thread transport
pino-roll	Rotating files

COMMON MISTAKES

- `log.info('user', user)` (console style); the second argument is ignored as a format arg
- Logging whole request objects; you leak cookies and auth headers
- String-building messages (`'took ' + ms`); keep values as fields so they're searchable

TIPS

- Log to stdout and let Docker / systemd / the cloud collect it
- Use `AsyncLocalStorage` to attach `requestId` automatically (Sheet 59)
- Lower to `debug` per module with child loggers when investigating

SUMMARY

- `pino` — fast JSON logger
- `log.info({ fields }, msg)` — call shape
- `child()` — contextual fields
- `redact` — keep secrets out

Running in Production

NODE_ENV=production · PM2 · systemd unit · health checks · graceful shutdown · reverse proxy

KEEPING A SERVER RUNNING

In production, something must **start** Node on boot, **restart** it when it crashes, pass **configuration**, collect **logs**, and stop it **gracefully**. On a Linux VM that's usually **systemd** (or PM2), with a reverse proxy like nginx or Caddy in front for TLS and static files. Containers and platforms (Sheet 67) do the same jobs for you.

NOTES

- Run as a dedicated, unprivileged user
- Logs from stdout go to `journalctl -u notes`
- Bind to `127.0.0.1` when a proxy sits in front

SYSTEMD SERVICE UNIT

```
# /etc/systemd/system/notes.service
[Unit]
Description=Notes API
After=network.target

[Service]
User=notes
WorkingDirectory=/srv/notes
EnvironmentFile=/srv/notes/.env
Environment=NODE_ENV=production
ExecStart=/usr/bin/node --enable-source-maps server.js
Restart=on-failure
RestartSec=2
TimeoutStopSec=15
LimitNOFILE=65535

[Install]
WantedBy=multi-user.target
```

CODE EXPLAINED

- → `EnvironmentFile` — secrets outside the repo
- → Absolute path to `node` — systemd has no nvm PATH
- → `Restart=on-failure` — restarts crashes, not clean stops
- → `TimeoutStopSec` — SIGTERM, then SIGKILL after 15 s
- → `LimitNOFILE` — room for many sockets

SYSTEMCTL

COMMAND	DOES
`daemon-reload`	Load unit changes
`enable --now notes`	Start + on boot
`restart notes`	Restart
`status notes`	State + recent logs
`journalctl -u notes -f`	Follow logs

PRODUCTION CHECKLIST

ITEM
`NODE_ENV=production`
`Graceful shutdown (19)`
`Health endpoints`
`Structured logs (65)`
`Error tracking`
`Non-root user`

HEALTH ENDPOINTS

PATH	ANSWERS
`/healthz`	Process alive?
`/readyz`	Ready for traffic (DB ok)?

COMMON MISTAKES

- `ExecStart=node server.js` with nvm-installed Node; systemd can't find it
- Running as root "to bind port 80"; use a proxy or `setcap` instead
- No restart policy; one crash means downtime until someone notices

TIPS

- Keep `/healthz` cheap; put dependency checks in `/readyz`
- Use `Environment=NODE_OPTIONS=--max-old-space-size=…` for heap limits
- Reload with zero downtime by running two instances behind the proxy

SUMMARY

- `systemd / PM2` — supervise the process
- `Restart=on-failure` — auto-recover
- `reverse proxy` — TLS and static files
- `health endpoints` — for monitors and LBs

Docker for Node

CONTAINERIZING A NODE APP

A good Node image installs **only production dependencies** from the lockfile, runs as the non-root **node** user, and starts **node directly** so signals reach your shutdown handler. A **multi-stage build** keeps build tools and dev dependencies out of the final image, and copying `package*.json` before the source lets Docker cache the install layer.

NOTES

- Official images include a `node` user (uid 1000)
- `-slim` is Debian with fewer packages; good default
- Alpine uses musl libc; some native modules misbehave

MULTI-STAGE DOCKERFILE

```
FROM node:26-slim AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev

FROM node:26-slim AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:26-slim
ENV NODE_ENV=production
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
COPY package.json ./
USER node
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

CODE EXPLAINED

- → `deps` stage — production-only `node_modules`
- → `build` stage — dev deps for TS / bundling, then discarded
- → `COPY package*.json` first — install layer is cached
- → `USER node` — don't run as root
- → Exec-form `CMD [...]` — node is PID 1 and gets SIGTERM

BASE IMAGES

TAG	NOTES
`node:26`	Full Debian, large
`node:26-slim`	Smaller, default pick
`node:26-alpine`	Smallest, musl libc
`distroless/nodejs`	No shell, hardened

PRACTICES

DO	WHY
`npm ci --omit=dev`	Small, exact deps
`.dockerignore`	Keep secrets out
`--init or tini`	Reap zombies, signals
`Pin a major tag`	Predictable builds

.DOCKERIGNORE

```
node_modules
.git
.env*
dist
*.log
```

COMMON MISTAKES

- `CMD npm start`; npm may not forward SIGTERM, so shutdown hangs then gets killed
- `COPY . .` before `npm ci`; every code change reinstalls dependencies
- Copying a local `node_modules` built for macOS into a Linux image

TIPS

- Add `HEALTHCHECK` or rely on your orchestrator's probes
- Scan images with `docker scout` or Trivy
- Set memory limits and size `--max-old-space-size` to match (Sheet 54)

SUMMARY

- `multi-stage` — small final image
- `npm ci --omit=dev` — production deps only
- `USER node` — non-root
- `CMD ["node", …]` — signals work

Single Executables & Bundling

SHIPPING WITHOUT NODE INSTALLED

A **Single Executable Application** (SEA) packs your app and the Node runtime into one binary that runs on machines with no Node installed, which is handy for CLIs and internal tools. Bundle your code into **one file** first (esbuild, Rollup), generate a SEA blob, and inject it into a copy of the `node` binary. **Bundling** alone also helps servers start faster and ship fewer files.

NOTES

- One binary per OS and CPU; build on each target
- macOS binaries must be re-signed after injection
- The SEA build steps have been changing; check the docs for your version

BUNDLE, THEN BUILD A SEA

```
# 1. bundle to one CommonJS file
npx esbuild src/cli.js --bundle --platform=node \
  --format=cjs --outfile=dist/cli.cjs

# 2. describe the executable
echo '{ "main": "dist/cli.cjs", "output": "sea.blob" }' > sea-config.json
node --experimental-sea-config sea-config.json

# 3. copy node and inject the blob
cp "$(command -v node)" notes
npx postject notes NODE_SEA_BLOB sea.blob \
  --sentinel-fuse NODE_SEA_FUSE_fce680ab2cc467b6e072b8b5df1996b2

./notes --help                          # runs without Node installed
```

CODE EXPLAINED

- → `--bundle` — inlines every dependency into one file
- → `--platform=node` — keeps `node:` modules as imports
- → `sea-config.json` — what goes into the blob
- → `postject` — writes the blob into the binary
- → Sentinel fuse — the marker Node checks at startup

SEA-CONFIG.JSON

FIELD	PURPOSE
`main`	Bundled entry file
`output`	Blob path
`assets`	Extra files to embed
`useCodeCache`	Faster startup
`disableExperimental… Warning`	Quiet startup

BUNDLERS

TOOL	NOTES
`esbuild`	Fastest, simple
`Rollup`	Libraries, tree-shaking
`tsdown`	TS libraries
`@vercel/ncc`	One-file Node apps

FASTER STARTUP

```
NODE_COMPILE_CACHE=.cache \
  node dist/server.js
```

COMMON MISTAKES

- Bundling native addons; `.node` files can't be inlined, ship them as assets
- Building on macOS and shipping to Linux; binaries aren't cross-platform
- Using `__dirname` paths to read files; inside a SEA they don't exist, use `sea.getAsset()`

TIPS

- Sign macOS and Windows binaries so users don't get security warnings
- Turn on the compile cache (`module.enableCompileCache()`) for faster cold starts
- Use `node --run` in scripts instead of `npm run` for quicker startup

SUMMARY

- `bundle` — one file, faster start
- `SEA` — app + Node in one binary
- `assets` — embed extra files
- `compile cache` — cache compiled code

Quick Reference — Runtime, Modules & Async

NODE CLI

COMMAND	DOES	#
`node app.js`	run a file	03
`node --watch app.js`	restart on change	03
`node --env-file=.env`	load env vars	20
`node --test`	run node:test	47
`node --inspect`	debugger	45
`node -e "code" / -p`	eval / print	03
`node app.ts`	run TypeScript (strip types)	05

MODULES

SYNTAX	DOES	#
`import x from "./x.js"`	ESM import	07
`export default / export const`	ESM export	07
`"type": "module"`	.js = ESM	07
`require("./x")`	CommonJS	06
`module.exports = …`	CJS export	06
`import.meta.dirname`	ESM `__dirname`	07
`node:` prefix	`node:fs`, `node:path`	07

NPM

COMMAND	DOES	#
`npm init -y`	package.json	09
`npm i pkg / -D pkg`	dep / dev dep	10
`npm ci`	clean install from lock	10
`npm run build`	run a script	10
`npx tool`	run without installing	10
`npm outdated / audit`	updates / vulns	57
`"engines": { "node": ">=22" }`	version guard	09

EVENT LOOP & TIMERS

API	DOES	#
`setTimeout(fn, ms)`	once later	13
`setInterval(fn, ms)`	repeat	13
`setImmediate(fn)`	after I/O phase	12
`process.nextTick(fn)`	before other callbacks	12
`queueMicrotask(fn)`	microtask	12
`await setTimeout(ms)`	`node:timers/promises`	13

ASYNC PATTERNS

API	DOES	#
`fs.promises / node:fs/promises`	promise APIs	15
`util.promisify(fn)`	callback → promise	15
`(err, data) => {}`	error-first callback	14
`Promise.all / allSettled`	run in parallel	16
`AbortController`	cancel work	16
`AbortSignal.timeout(ms)`	time limit	16

EVENTEMITTER

API	DOES	#
`class X extends EventEmitter`	custom emitter	17
`on(evt, fn) / once`	listen	17
`emit(evt, …args)`	fire	17
`off(evt, fn)`	remove listener	17
`"error" event`	throws if unhandled	17
`events.once(em, "x")`	await an event	17

PROCESS

API	DOES	#
`process.argv`	CLI args	22
`process.env.PORT`	env var	20
`process.exit(code)`	quit	18
`process.exitCode = 1`	exit after drain	18
`process.cwd()`	working dir	18
`process.on("SIGTERM", fn)`	graceful shutdown	19
`process.memoryUsage()`	memory stats	54

ARGS, INPUT & OS

API	DOES	#
`util.parseArgs({ options })`	parse flags	22
`readline/promises question()`	prompt user	23
`process.stdin`	piped input	23
`console.table / time`	output helpers	24
`util.styleText("red", s)`	colored text	24
`os.cpus() / os.EOL`	system info	21

⚠ COMMON MISTAKES

- Mixing ESM and CJS without knowing which one a file is — check `"type"` and the extension
- Blocking the event loop with sync I/O or heavy CPU work on a server
- Unhandled `"error"` events or rejections crashing the process

✓ TIPS & BEST PRACTICES

- Use `node --watch` and `--env-file` instead of extra tools in dev
- Pass an `AbortSignal` through long operations so they can be cancelled
- Set `process.exitCode` instead of calling `process.exit()` mid-write

QUICK SUMMARY

- ESM with `"type": "module"` and `node:` imports
- `fs/promises` + `async` / `await` everywhere
- `nextTick` → microtasks → timers → I/O → `setImmediate`
- `process.env`, `process.argv`, signals for lifecycle

Quick Reference — Files, Network & Tooling

FS/PROMISES

API	DOES	#
`readFile(p, "utf8")`	read text	26
`writeFile(p, data)`	write / replace	27
`appendFile(p, data)`	append	27
`mkdir(p, { recursive: true })`	make dirs	28
`readdir(p) / glob("**/*.js")`	list / glob	28
`stat(p) / rm(p, { recursive })`	info / delete	29
`fs.watch(p)`	watch for changes	29

PATH

API	DOES	#
`path.join(a, b)`	join segments	30
`path.resolve(p)`	absolute path	30
`path.basename / dirname`	name / folder	30
`path.extname(p)`	extension	30
`path.parse(p)`	split into parts	30
`fileURLToPath(url)`	URL → path	30

BUFFERS

API	DOES	#
`Buffer.from("hi", "utf8")`	text → bytes	31
`buf.toString("base64")`	encode	31
`Buffer.alloc(16)`	zero-filled	31
`Buffer.concat([a, b])`	join	31
`buf.readUInt32BE(0)`	binary read	32
`TextEncoder / TextDecoder`	web APIs	32

STREAMS

API	DOES	#
`fs.createReadStream(p)`	read in chunks	34
`fs.createWriteStream(p)`	write in chunks	34
`pipeline(a, b, c)`	pipe + error handling	35
`new Transform({ transform })`	change chunks	35
`for await (const c of stream)`	iterate	34
`Readable.toWeb()`	Node ↔ web streams	36

HTTP & FETCH

API	DOES	#
`http.createServer((req, res) => …)`	server	37
`res.writeHead(200, h); res.end()`	respond	37
`new URL(req.url, base)`	parse URL	38
`await fetch(url)`	HTTP client	39
`res.json() / res.ok`	read response	39
`WebSocket (global)`	WS client	41

ERRORS

ITEM	DOES	#
`err.code`	ENOENT, EADDRINUSE…	43
`new Error(msg, { cause })`	wrap errors	43
`uncaughtException`	last-resort handler	44
`unhandledRejection`	crash by default	44
`--trace-warnings`	warning stacks	45
`--cpu-prof / --heap-prof`	profiles	46

NODE:TEST & ASSERT

API	DOES	#
`test("name", () => {})`	define a test	47
`describe / it`	grouping	47
`assert.strictEqual(a, b)`	equality	48
`assert.deepStrictEqual`	deep equality	48
`await assert.rejects(p)`	expect rejection	48
`mock.fn() / mock.timers`	mocks	49
`--experimental-test-coverage`	coverage	49

CONCURRENCY & SECURITY

API	DOES	#
`child_process.execFile`	run a program	50
`spawn(cmd, args)`	stream output	50
`new Worker("w.js")`	worker thread	51
`cluster / PM2`	multi-process	52
`crypto.randomUUID()`	UUID	56
`crypto.createHash("sha256")`	hashing	56
`--permission`	permission model	55

⚠ COMMON MISTAKES

- `exec` with user input in the command string (shell injection) — use `execFile` / `spawn`
- `pipe()` without error handling — use `stream.pipeline`
- Reading huge files with `readFile` instead of streaming them

✓ TIPS & BEST PRACTICES

- Build paths with `path.join`, never string concatenation
- Use the built-in `fetch`, `node:test` and `--watch` before adding dependencies
- Handle `SIGTERM`: stop accepting requests, finish in-flight ones, then exit

QUICK SUMMARY

- `fs/promises` + `path` for files
- Streams + `pipeline()` for big data
- `http.createServer` serves, `fetch` requests
- `node:test` + `assert` for tests